Sixfold and sevenfold rotationally symmetric point sets without a convex octagon

[author]

Draft P1, 6 November 2026. Not refereed.

Abstract

Let $ES(k)$ be the least n such that every n points in general position in the plane contain k points in convex position. Erdős and Szekeres conjectured $ES(k) = 2^{k-2} + 1$; this is known for $k \leq 6$ and open for $k = 7$ and $k = 8$. We study point sets that are invariant under a rotation of order m. For $k = 8$ we show that the largest m-fold rotationally symmetric set in general position with no 8 points in convex position has exactly 60 points for $m = 6$ and exactly 64 points for $m = 7$. Sixfold symmetric sets in general position have $6t$ points (or consist of the centre alone) and sevenfold ones $7t$ or $7t + 1$, so this says that $ES(8) = 65$ holds within these two symmetry classes and that the largest sizes it permits there are attained. The proof is computer-assisted. A reduction checked in Lean 4 brings the upper bounds down to invariant sets of exactly 66, resp. 70, points without the centre; by a layer lemma (Lean-checked core, written assembly) these are 11 nested regular hexagons, resp. 10 nested regular heptagons, in general position without a convex octagon. Two separately implemented encoders turn these statements into CNF formulas; the soundness of the encodings is a written proof (not refereed), and the formulas are refuted by CaDiCaL with LRAT proofs accepted by the verified checker `cake_lpr`. For the sixfold bound, a hash-bound `cake_lpr` replay (6 November 2026) solved all 64 cube formulas of the original split again, after regenerating them with hashes matching the record, and `cake_lpr` accepted every fresh proof. For one of the encoders, its 32 replayed cube formulas and 32 further ones, certified the same day, cover all 2^6 sign patterns of six phase-order variables, so that encoder's sixfold base formula is certified unsatisfiable on its own and its route to the sixfold bound does not use the reflection symmetry breaking. For this encoder the sixfold bound is moreover formally verified end to end in Lean 4 with Mathlib, from the unsatisfiability of the 64 cube formulas as printed by a Lean program to the geometric statement, assuming the 64 `cake_lpr` verdicts and six stated trust items (among them that the certified files are these printed formulas, which is checked by their hashes, and that `cake_lpr` parses them as the Lean parser does); the sevenfold bound rests on the written soundness proof. The lower bounds are explicit sets with coordinates in $\mathbb{Z}[\zeta_6]$ and $\mathbb{Z}[\zeta_7]$, five sixfold and seven sevenfold order types, checked by exact computation with separately written programs. Every statement carries a rigor label. The half-turn and the threefold, fourfold and fivefold cases remain open.

Contents

1 Introduction **2**
1.1 Related work and credit 5
1.2 Organisation 5

2 Definitions, notation and rigor labels **5**

3 The layer reduction (Lean-checked core) **6**

4 The encodings **8**

5 Soundness of the encodings for the $k = 8$ rotational upper bounds **10**
5.1 Configurations and layers 10
5.2 Phases, genericity and the intended assignment 12

5.3 Triple orbits and orientation variables . . . 14
5.4 Directions and the angular classification . . . 15
5.5 Phase-order, nesting and chain clauses . . . 17
5.6 The reflection unit . . . 18
5.7 The soundness theorem . . . 18
5.8 The omitted antipodal case . . . 20
5.9 The cube cover and the passage to geometry . . . 20
5.10 Trust base and gaps . . . 22

6 The certificates **24**
6.1 Sevenfold . . . 25
6.2 Sixfold . . . 25
6.3 The upper bounds . . . 26

7 The lower-bound constructions **27**

8 Trust and rigor **29**

9 Reproducibility **32**

10 Open problems **33**

A Hashes **34**

B Seeds of the point sets **35**

1 Introduction

The problem. Let $ES(k)$ be the least n such that every set of n points in general position in the plane contains k points in convex position. Erdős and Szekeres proved that $ES(k)$ exists [9], constructed 2^{k-2} points in general position without k points in convex position [10], and conjectured $ES(k) = 2^{k-2} + 1$. The conjecture holds for $k \leq 6$: $ES(4) = 5$ (E. Klein, see [9]), $ES(5) = 9$ (Makai, as reported in [9, p. 464], and Makai and Turán according to [10]; unpublished; published proof by Kalbfleisch, Kalbfleisch and Stanton [15]) and $ES(6) = 17$, proved by Szekeres and Peters with a computer search [31] (Marić [19], with a formally verified reduction, and Scheucher [23] independently verified it by SAT; faster SAT proofs are in [24, 13], and a Lean-verified encoding gives $ES(6) \leq 17$ as a corollary [27, §7]). It is open for $k = 7$ and $k = 8$. The best refereed upper bound for $k = 8$ that we know of is $ES(8) \leq 408$ [22]. Unrefereed bounds are $ES(8) \leq 319$ in Schroeder's preprint [25] and $ES(8) \leq 311$ in separate work of this project, which is checked by the Lean kernel in two complete runs (the second a reproduction with a compact certificate checker) and built on Schroeder's omission method and catalogs (`NOVELTY20261106`, `experiments/omission-final311-2026-10-14`, `experiments/omission-compact-checker-2026-11-05` [30]). Asymptotically $ES(k) = 2^{k+o(k)}$ by Suk [29], with an improved error term in [14].

Symmetric point sets. Imposing a symmetry shrinks the search space, and optimality may be easier to prove for structured sets. Subercaseaux, Mackey, Qian and Heule (SMQH) [28] built rotational symmetry into SAT encodings, combined them with a local-search realizer, and found fourfold and fivefold symmetric 16-point sets without a convex hexagon [28, §6.1, App. C] (see also Mackey's thesis [18]); for 32 points without a convex heptagon they report that no realizable fourfold symmetric set exists [28, §6.2]. They wrote [28, §7]: "for the Erdős–Szekeres problem, we suspect that there are symmetric constructions for all $k \geq 7$, and proving this could represent progress toward the conjectured bound $g(k) = 2^{k-2} + 1$, since it might be easier to prove optimality for more structured solutions." Their results stop at $k = 7$. This paper treats $k = 8$ for the two rotation orders for which we could settle both directions, $m = 6$ and $m = 7$.

Results. For $m \geq 2$ and $k \geq 3$ let $N_m(k)$ be the largest number of points of a finite set in general position in the plane that is invariant under the rotation by $2\pi/m$ about some point and contains no k points in convex position.

Theorem 1.1 (main theorem; rigor as stated below)**.** *$N_6(8) = 60$ and $N_7(8) = 64$. In detail:*

(a) (upper bounds) *every set of at least 61 points in general position that is invariant under a rotation by $2\pi/6$ contains 8 points in convex position, and so does every set of at least 65 points in general position that is invariant under a rotation by $2\pi/7$;*

(b) (lower bounds) *there are sixfold symmetric sets of 60 points (ten nested regular hexagons) and sevenfold symmetric sets of 64 points (nine nested regular heptagons and the centre) in general position without 8 points in convex position; they include at least five, resp. seven, pairwise different order types.*

Rigor. *Part (a) is computer-assisted: unsatisfiability certificates (`cake_lpr`) for two separately implemented encoders (Section 6). For $m = 7$ both certificates were replayed against regenerated formulas with matching hashes. For $m = 6$ all 64 cube formulas of the original split (32 per encoder) were regenerated with matching hashes, solved again and certified in a hash-bound `cake_lpr` replay (6 November 2026), 64 of 64 accepted. For encoder B the sixfold base formula itself is certified unsatisfiable by 64 hash-bound `cake_lpr` verdicts, one for each sign pattern of the six phase-order variables $Y(1,2), \ldots, Y(3,4)$ (Section 4): 32 from that replay and 32, for the cubes with $\neg Y(1,2)$, from a run of the same day (32 of 32 accepted); so the sixfold bound via encoder B does not use the reflection lemma (Lemma 5.23). For both m this rests on a Lean-checked reduction (Section 3) and a written soundness proof of the encodings, not refereed (Section 5), whose remaining gaps are listed as (G1)–(G4) in Section 5.10. For $m = 6$ there is a second route, via encoder B, that uses neither the written soundness proof nor the Python generator: the Lean theorem `EncB.Geo.sixfold_es8_of_texts` (Section 8, "Encoder B in Lean") proves part (a) for $m = 6$ from the hypothesis that the 64 cube formulas printed by a Lean program are unsatisfiable, which is what the 64 hash-bound `cake_lpr` verdicts certify. So part (a) for $m = 6$ is formally verified end to end via encoder B, assuming these 64 `cake_lpr` verdicts and the trust items (E1)–(E6) of Section 5.10 (gap (G2)); part (a) for $m = 7$ rests on the written proof. Part (b) is exact computation (Section 7).*

Corollary 1.2 ($ES(8) = 65$ within two symmetry classes)**.** *Every set of at least 65 points in general position that is invariant under a rotation of order 6 or 7 contains 8 points in convex position. For $m = 6$ the largest size below 65 that an m-fold symmetric set in general position can have is 60, for $m = 7$ it is 64, and both sizes are attained by sets without 8 points in convex position. (Rigor: Theorem 1.1 together with the short argument below, a written proof, not refereed, which is new in this draft and has had no internal review. For order 6 the first sentence also follows from the Lean theorem `EncB.Geo.sixfold_es8_order6_of_texts`, which states it in the stronger form with 61 points, for invariance under the rotation by $2\pi/6$ or by $2\pi \cdot 5/6$, with the trust of Theorem 1.1(a) for $m = 6$ via encoder B; the sizes 60 and 64 are not covered by it.)*

Proof (written proof, not refereed; new). A rotation ρ' of order m about a point O is the rotation by $2\pi j/m$ about O for some j with $\gcd(j, m) = 1$. If $jj' \equiv 1 \pmod m$, then ${\rho'}^{j'}$ is the rotation ρ by $2\pi/m$ about O, and a set invariant under ρ' is invariant under all powers of ρ', so under ρ; Theorem 1.1 applies. A set invariant under ρ is a union of ρ-orbits, each of m points except the centre O. If m is even and the set is in general position and contains a point $p \neq O$, it does not contain O, since p, O and $\rho^{m/2}p = 2O - p$ are collinear (Lean-checked, `centre_not_mem_of_even`; Lemma 5.6). So sixfold sets in general position have $6t$ points (or are $\{O\}$), and sevenfold ones $7t$ or $7t + 1$. The largest such numbers below 65 are 60 and $64 = 7 \cdot 9 + 1$. The rest is Theorem 1.1. □

So $N_6(8) = 60$ is not a separate phenomenon: 60 is the largest multiple of 6 below 65, and the content of the sixfold statement is the sixfold case of $ES(8) = 65$ together with attainment. The same holds for $m = 7$ and 64.

Remark 1.3 (affine symmetries; written proof (not refereed), new in this draft). The same values hold for sets invariant under an affine bijection g of order 6, resp. 7. Indeed g permutes the set, which contains three non-collinear points, so a power of g fixes three non-collinear points and is the identity; averaging a scalar product over the finite group $\langle g \rangle$ gives a linear map T with $\det T > 0$ such that TgT^{-1} is a Euclidean isometry of the same order. An isometry of finite order at least 3 is a rotation by $2\pi j/m$ with $\gcd(j, m) = 1$, whose powers include the rotation by $2\pi/m$, and T preserves general position and convex position. The

conjugation of an affine symmetry to an isometry is Lean-checked inside `hasConvexNGon_of_affine_symmetry` (`SYMLEAN/AffineReduction.lean` [30]), but the statement of that theorem quantifies over all non-identity Euclidean isometries and does not track the order of the symmetry; an order-preserving version, which this remark needs, is not formalized.

m	sizes	largest known	needed for 65 points	formula at threshold	status
2	$2t$	64 (32 orbits)	33 orbits	— (layer lemma fails)	open
3	$3t$, $3t+1$	61 (20 orbits, centre)	22 orbits, no centre	— (layer lemma fails)	open
4	$4t$	52 (13 layers)	17 layers	130,439 var., 4,658,852 cl.	open
5	$5t$, $5t+1$	56 (11 layers, centre)	13 layers	91,390 var., 3,295,617 cl.	open
6	$6t$	**60** (10 layers)	11 layers	78,059 var., 2,387,083 cl.	excluded; max. 60
7	$7t$, $7t+1$	**64** (9 layers, centre)	10 layers	80,080 var., 2,748,538 cl.	excluded; max. 64
≥ 8	mt, $mt+1$	1 (the centre)	—	—	an orbit is a convex m-gon

Table 1: Rotational symmetry of order m and convex octagons: sizes of m-fold symmetric sets in general position (for even m also the set consisting of the centre alone; in row ≥ 8, $mt+1$ only for odd m), the largest known set without 8 points in convex position, and the number of layers or orbits a counterexample to $ES(8) = 65$ would need. Lower bounds: exact computation (`ES8SYM`, `SIX60`, `PRIMEFOLD` [30]); for $m = 2$ also implied by Baek and Balko's blow-up lemma [2, Lemma 14] applied to the 16-point sets of [28, 18] (Section 1.1). Rows 6 and 7: Theorem 1.1. Row ≥ 8: elementary; a version over $\mathbb{C}$ with centre 0 is Lean-checked as `RefuteC6.part_a` (`ES8SYM/lean/C6check.lean`), whose link to the conventions of Section 2 is not formalized. The formula sizes are those of encoder A (Section 4); for $m = 2$ and $m = 3$ the layer lemma does not apply ($8 > 2m$). Rows 2–5 are discussed in Section 10; for them the best refereed upper bound comes from $ES(8) \leq 408$ [22] (unrefereed bounds on $ES(8)$ are cited in Section 1).

Method. For $m \geq 3$ and $K \leq 2m$, the convex layers of an m-fold symmetric set in general position without K points in convex position are regular m-gons about the centre, strictly nested (the *layer lemma*, Lemma 3.1, whose core is Lean-checked and whose assembly into layers is a written step; for 16 points, fourfold symmetry and $K = 6$ this is the special case used by SMQH [28, §3.2]). The reduction of part (a) of Theorem 1.1 to two statements about invariant sets of exactly 66 and exactly 70 points without the centre (`Sixfold66` and `Sevenfold70`, Proposition 3.3) is Lean-checked; that such sets are 11 nested regular hexagons, resp. 10 nested regular heptagons, is Lemma 5.2 (Lean-checked core, written assembly). The orientation of a triple of points of such a configuration is determined, in many cases, by the circular order of the directions of the points and their antipodes, which depends only on the relative phases of the layers; together with nesting facts and a clause family that forbids convex octagons through chains around a pivot, this gives a CNF formula (Section 4). Section 5 proves that every real configuration yields a model; the formulas are refuted with LRAT proofs checked by `cake_lpr` (Section 6). The lower bounds were found by search (alternating projections between real configurations and SAT models, a phase-fixed linear-programming loop, simulated annealing and cluster constructions) and are verified in exact arithmetic (Section 7).

What is and is not claimed. The theorem is computer-assisted and has not been refereed. Its weakest links are the written parts of the soundness argument, in particular that the Python generators emit exactly the analysed clause families (gap (G1) of Section 5.10); for $m = 7$ the written proof is the only route. For $m = 6$ there is a second route via encoder B, Lean-checked end to end up to the trust items listed below (Section 8, "Encoder B in Lean"): a Lean definition of encoder B's formula, the step from a sixfold configuration without a convex octagon to a model of that formula (with Mathlib), the soundness of the clauses for that model, the 64-cube cover, the reading of the printed texts by a DIMACS parser defined in Lean, and the reduction of Section 3 are composed into one Lean theorem, `EncB.Geo.sixfold_es8_of_texts`, whose only hypothesis besides those of Theorem 1.1(a) is that the 64 cube texts printed by a Lean program parse to unsatisfiable clause lists. So the sixfold bound is formally verified end to end, via encoder B, assuming the 64 `cake_lpr` verdicts and the trust items (E1)–(E6) of Section 5.10: that the certified files are these printed texts (checked by their SHA-256, by running the compiled printer, not proved), the verdicts of `cake_lpr`, that `cake_lpr` parses the files as the Lean parser does (tested, not verified), the Lean compiler for the printer

and a reading of its input/output code, the Lean kernel and Mathlib with the match of the Lean definitions to the notions of this paper, and SHA-256. The sevenfold bound, encoder A (the cross-check) and the lower bounds are not covered by this route. Every `cake_lpr` verdict that the upper bounds use is bound to the hashes of its formula and proof files (Section 6). The proof files are archived outside the repository and are not yet public. No statement here is a priority claim.

1.1 Related work and credit

Computer-assisted Erdős–Szekeres-type results. Besides the works cited above [31, 19, 23, 24, 13], Balko and Valtr [3] ran a SAT attack on the Erdős–Tuza–Valtr refinement [11] of the conjecture; Subercaseaux et al. verified the encoding of the empty hexagon theorem in Lean 4 and checked its proof with `cake_lpr` [27], a precedent for the combination of Lean and verified proof checking used here (our encoder soundness is formalized only for the sixfold bound with encoder B); Krapivin, Przybocki and Heule study satisfiability modulo realizability [17]; and Dumitru reports SAT computations for the 33-point problem ($k = 7$) on anchored subfamilies [8]. One of Dumitru's convex-layer templates implies the fivefold case of the analogous statement for $k = 7$, with priority [8, §7, Table 1]; we found no statement on symmetric sets for $k \geq 8$ in [8]. Georgiev, Gómez-Serrano, Tao and Wagner give 2^{k-2}-point sets without a convex k-gon for $k \leq 8$, with no symmetry stated [12, §38].

Symmetric sets. Rotational symmetry in SAT encodings for Erdős–Szekeres-type problems, and the special case of the layer lemma noted above, are due to SMQH [28] and Mackey [18]; our encodings likewise build the rotation into the variables, with one variable per rotation orbit of triples. Symmetric or almost symmetric extremal constructions for variants of the problem appeared earlier [21, 23, 26] (as credited in [28, §1]). For the other symmetry types at $k = 8$ the following is known to us. For reflections, the largest octagon-free set has 46 points; this follows by an elementary mirror lemma, as shown in a companion note [30, `WRITEUP/note/symmetric_extremal.tex`], from Baek's theorem [1, Thm. 1.6] that every set of $\binom{n-1}{2} + 2$ points in general position with distinct x-coordinates contains a 4-cap (four points with increasing x-coordinates and decreasing slopes) or n points in convex position. For the half-turn, Baek and Balko's blow-up lemma [2, Lemma 14] (whose proof is omitted in the conference version; the journal version was not read) applied to the 16-point sets of [28, 18] gives 64-point centrally symmetric octagon-free sets; the same note also gives a construction that does not use Lemma 14, and the explicit 64-point sets are checked by exact computation. These are not results of this paper.

What we did not find. In the sources we inspected we did not find prior work on rotationally symmetric point sets without 8 points in convex position, or on the values $N_6(8)$ and $N_7(8)$. The inspection covered SMQH [28] (arXiv version 1; the CICM chapter text was not accessed), Mackey's thesis [18], [8, 12, 17, 16, 1, 2], the survey [21], arXiv listings and full-text searches, Zenodo, GitHub and general web searches on 21 October and 1–2, 3, 4 and 6 November 2026 (records: `LIT`, `PRIMEFOLD` §8, `C6UPPER/README.md`, `C6C7REVIEW/LITERATURE.md`, `NOVELTY20261106` [30]). The symmetric literature we found stops at $k = 7$. A bounded search can miss work, in particular journal versions, manuscripts and non-English sources; this is not a priority claim.

1.2 Organisation

Section 2 fixes notation and rigor labels. Section 3 gives the layer reduction (Lean-checked core), Section 4 the encodings, and Section 5 the proof that they are sound. Section 6 describes the certificates and Section 7 the lower-bound constructions. Section 8 collects the rigor of every ingredient, Section 9 the reproduction data, and Section 10 open problems. Appendix A lists hashes and Appendix B generators of the point sets.

2 Definitions, notation and rigor labels

Points are elements of $\mathbb{R}^2$. For points p, q, r let

$$\mathrm{orient}(p, q, r) = (x_q - x_p)(y_r - y_p) - (y_q - y_p)(x_r - x_p),$$

twice the signed area of the triangle pqr; it is positive if p, q, r turn counterclockwise. A finite set is in *general position* if no three of its points are collinear, and in *convex position* if no point lies in the convex hull of the others. For $K \geq 3$ a set is *K-free* if none of its K-element subsets is in convex position; a *convex K-gon* of a set is a K-subset in convex position. Let O be the origin and $\rho = \rho_m$ the counterclockwise rotation by $2\pi/m$ about O. A set S is *m-fold (rotationally) symmetric* if $\rho S = S$ after translating its centre to O; its *orbits* are the sets $\{\rho^j x\}$, regular m-gons for $x \neq O$. $N_m(k)$ is defined in Section 1.

Lean conventions. In Lean a point is `Point = Fin 2 → ℝ`; the predicates `GeneralPosition`, `ConvexPosition` (Mathlib's `ConvexIndependent` of the inclusion) and `HasConvexNGon n S` are those of the repository's `ErdosSzekeres.Definitions`; `rotAbout c a` is the rotation by the angle a about c, and `Invariant f S` means $f(p) \in S$ for all $p \in S$. Section 5.1 relates these to the notions above.

Rigor labels. Every statement carries one of the following labels.

- *Lean-checked*: compiled with Lean 4.24.0 [6] and Mathlib [20] at commit `f897ebcf` (for the encoder-B package `ENCBLEAN` of Section 8: its part `ENCBLEAN/EncB` with Lean 4.24.0 and its standard library only, without Mathlib, and its part `ENCBLEAN/geo`, the geometric half and the assembly, with Lean 4.24.0 and Mathlib at the same commit `f897ebcf`), with no `sorry`, `admit` or added axioms; `#print axioms` reports only `propext`, `Classical.choice` and `Quot.sound`.
- *Certificate*: unsatisfiability of a specific CNF file, established by an LRAT proof [5] that the verified checker `cake_lpr` [32] accepted with the line `s VERIFIED UNSAT`. The solver (CaDiCaL 3.0.1 [4], and 2.1.3 for the re-checks of 5 November, the sixfold replay of 6 November and the run for encoder B's cubes with $\neg Y(1,2)$ of the same day) is not trusted. Acceptance by `lrat-check` alone is not a certificate: the `lrat-check` of drat-trim at commit `2e3b2dc` accepts an invalid proof of a satisfiable formula unless patched, a publicly known defect [7] (our reproduction: `LRATCHECK20261105`), so its verdicts are cited as supporting only. Where a `cake_lpr` verdict is on record only as a solve-time log line naming a cube label, without formula or proof hashes, this is said each time it is mentioned; since the replay of 6 November the upper bounds do not depend on such verdicts. A certificate proves a geometric statement only together with the soundness of the encoding (written for both encoders; for the sixfold bound via encoder B also Lean-checked, Section 8).
- *Exact computation*: integer or exact algebraic arithmetic on explicit data, by the programs named.
- *Written proof (not refereed)*: a complete written proof, here or in the supplementary material; where it was internally reviewed, this is said.
- *Evidence only*: tests, timings and failed searches; never used as a proof.

Supplementary material. Paths such as `C6UPPER` refer to directories under `cloud/es7-2026-10-19/results` of the project repository [30] (paths beginning with `experiments/`, to its root), which contain the programs, logs, point sets, Lean files, hashes and reproduction commands. Proof files are in local archives outside the repository and are not public; an archived public deposit of the supplementary material with a DOI is an open item of this draft.

3 The layer reduction (Lean-checked core)

The Lean statements of this section are in `LAYERLEAN/LayerLemma.lean` (namespace `ErdosSzekeres.LayerLemma`); where a statement combines them by a written step, this is said.

Lemma 3.1 (layer lemma; Lean-checked core (`layers_are_orbits`, `exists_layer`, `strict_nesting`, `centre_not_mem_of_even`); the assembly into layers is a written proof (not refereed))**.** *Let $m \geq 3$, $K \leq 2m$, and let S be a finite ρ-invariant set in general position with no K points in convex position. Then $S \setminus \{O\}$ is the union of regular m-gons $L_0, L_1, \ldots, L_{t-1}$ centred at O (its convex layers), and for $n < n'$, every vertex x of L_n and every $y \in L_{n'}$ satisfy* $\operatorname{orient}(x, \rho x, y) > 0$. *If m is even and $S \neq \{O\}$, then $O \notin S$.*

Proof sketch. The set of hull vertices of $S \setminus \{O\}$ is ρ-invariant and nonempty, so it is a union of orbits of exactly m points each; two orbits would give $2m \geq K$ points in convex position, so it is one orbit, a regular

m-gon. Peel it off and repeat. For strict nesting, with $\tau = \pi/m$,

$$\operatorname{orient}(x, \rho x, \rho^j x) = 4\,|x - O|^2 \sin\tau\, \sin j\tau\, \sin(j-1)\tau \;\geq 0,$$

so the vertices of a layer, and by convexity its hull, lie weakly to the left of each edge; equality would give three collinear points of S. For even m, $\rho^{m/2}$ is the half-turn, and $p, O, 2O - p$ are collinear. □

In Lean the core statement `hullVerts_eq_orbitFS` is proved for an injective affine map whose points all have minimal period m, without general position; `hullVerts_eq_orbitFS_rotAbout` specialises it to rotations, `layers_are_orbits` and `exists_layer` give the layers, `card_peel` their sizes, `strict_nesting` the nesting (it needs general position and $m \geq 2$), and `centre_not_mem_of_even` the centre exclusion. Combining these into the decomposition of $S \setminus \{O\}$ into layers $L_0, \ldots, L_{t-1}$ is a short written step, not formalized as one theorem. The version used by the soundness proof, for sets with exactly mt points and their peeled layers, is Lemma 5.2; there the assembly is written out, and it is listed as unreviewed under gap (G4). For even m and sets of exactly mt points, the part of this step that the Lean route for encoder B of Section 8 uses, a polar labelling of the peeled layers, is Lean-checked (`EncB.Geo.labelling_of_counterexample` in `ENCBLEAN/geo`).

Polar form (Lean-checked, `orient_polar`). Writing the points as $O + r(\cos\theta, \sin\theta)$,

$$\operatorname{orient}(a, b, d) = r_a r_b \sin(\theta_b - \theta_a) + r_b r_d \sin(\theta_d - \theta_b) + r_d r_a \sin(\theta_a - \theta_d).$$

Lemma 3.2 (encoding facts; Lean-checked)**.** *Under the hypotheses of Lemma 3.1:*

(a) (rule A) *if the three angular gaps from a to b to d lie in $(0, \pi)$, so that O lies inside the triangle, then* $\operatorname{orient}(a, b, d) > 0$ *(*`orient_polar_centre_inside`*);*

(b) (rule B) *if $\theta_b = \theta_a + x$ and $\theta_d = \theta_b + y$ with $x, y > 0$, $x + y < \pi$, and $r_b \geq r_a, r_d$, then* $\operatorname{orient}(a, b, d) > 0$ *(*`orient_polar_halfplane`*, using* $\sin x + \sin y > \sin(x + y)$*,* `sin_add_lt`*);*

(c) (radius order) *points of later layers are no farther from O than points of earlier layers (*`dist2_le_of_later`*);*

(d) (chains) *if $4 \leq K \leq 2m$, there are no pivot $p \in L_i$ and points $q_0, \ldots, q_{K-2}$ of $L_i \cup L_{i+1} \cup \cdots$, all other than p, with* $\operatorname{orient}(p, q_s, q_{s+1}) > 0$ *and* $\operatorname{orient}(q_s, q_{s+1}, q_{s+2}) > 0$ *for all admissible s (*`chain_clauses_sound`*);*

(e) (reduction) *if every ρ-invariant set in general position with exactly mt points and without O has a convex K-gon, then so does every ρ-invariant set in general position with more than $m(t-1)$ points other than O (*`convexNGon_of_layers`*).*

Proof sketch. (a) All three sines are positive. (b) By $r_b \geq r_a, r_d$ the polar form is at least $r_a r_d(\sin x + \sin y - \sin(x + y)) > 0$. (c) By convexity of the disc and Lemma 3.1. (d) *Fan lemma* (`hasConvexNGon_of_fan`): a pivot and $n \geq 3$ chain points in an open half-plane through the pivot, with all these turns positive, are in convex position. *Half-plane* (`lin_pos_of_peel`): every other point of the layers $\geq i$ lies strictly on the inner side of the tangent line at p; for $\rho^j p$ the value is $|p - O|^2(1 - \cos(2\pi j/m))$, and for later layers it follows from strict nesting at the two edges through p. So a chain would be a convex K-gon. (e) Peel t layers of exactly m points (`card_peel`). □

The two statements that the certificates establish are, verbatim:

```
def Sixfold66 : Prop :=
  ∀ (T : Finset Point) (c : Point), GeneralPosition T → c ∉ T →
    Invariant (rotAbout c (2 * Real.pi / 6)) T → T.card = 66 → HasConvexNGon 8 T
def Sevenfold70 : Prop :=
  ∀ (T : Finset Point) (c : Point), GeneralPosition T → c ∉ T →
    Invariant (rotAbout c (2 * Real.pi / 7)) T → T.card = 70 → HasConvexNGon 8 T
```

Proposition 3.3 (reduction; Lean-checked)**.** `sixfold_octagon`*: if* `Sixfold66` *holds, then every set S in general position with* `Invariant (rotAbout c (2 * Real.pi / 6)) S` *and $|S| \geq 61$ satisfies* `HasConvexNGon 8 S`*.* `sevenfold_octagon`*: if* `Sevenfold70` *holds, the same holds with $2\pi/7$ and $|S| \geq 65$.*

Proof sketch. For $m = 6$ the centre is not in S (Lemma 3.1), so $|S| \geq 61 > 6{\cdot}10$; for $m = 7$, $|S \setminus \{c\}| \geq 64 > 7{\cdot}9$. Apply Lemma 3.2(e) with $t = 11$, resp. $t = 10$. Lowering 61 to 60, resp. 65 to 64, makes the Lean proof fail (reported in `LAYERLEAN/README.md`; no log of these runs was kept). □

Build and axioms. `LAYERLEAN/LayerLemma.lean` has SHA-256 `afb8542d...` (Appendix A). It is checked with `lake env lean` from the repository root (Lean 4.24.0, Mathlib `f897ebcf`); the recorded run exits 0 with no messages (`LAYERLEAN/logs/build.log`), and `LAYERLEAN/logs/axioms.log` reports only `propext`, `Classical.choice`, `Quot.sound` for every theorem cited except the one-line `hullVerts_subset`, which is used in `layers_are_orbits` and `exists_layer` and so is covered by their checks. For this draft the file was checked again on 6 November 2026 with the already-built dependencies and 19 `#print axioms` lines appended (the declarations of Lemmas 3.1, 3.2 and Proposition 3.3, among them `fan_positive`): exit status 0, the three standard axioms for all 19, 23 s wall clock and 2.6 GB peak memory (`LAYERLEAN/logs/axioms_recheck_20261106`).

What is not in Lean. The assembly of the layer decomposition from the cited theorems (Lemmas 3.1 and 5.2), the classification of phase patterns (which of rules A and B applies, and to which ordered triple), the perturbation to generic phases, the consistency of one variable per triple orbit, the correspondence between the generator programs and the clause families, the reflection unit (used only by encoder A, the cross-check), the cube cover and the passage from unsatisfiable DIMACS files to `Sixfold66` and `Sevenfold70`. These are the written proofs of Section 5. For encoder B with $m = 6$ (so not for the sevenfold formulas), the separate package `ENCBLEAN` (Section 8, "Encoder B in Lean") has a Lean counterpart of every step of this list that the sixfold route via encoder B needs, for a Lean definition of its formula whose printed texts have the hashes of the certified files: the labelling of the layers, the geometric facts behind the clauses (including the sign rules on the circular order and the distinct phases), the correspondence between the clauses and a combinatorial model, the cube cover and the passage to `Sixfold66`, composed into one Lean theorem. That route needs neither the reflection unit nor a perturbation. `gen_b2.py` itself is not Lean-checked, and the Lean route does not use it.

4 The encodings

Fix (m, K, t); the certified instances are $(6, 8, 11)$ and $(7, 8, 10)$. Label the points of t nested regular m-gons by $\Lambda = \{0, \dots, t-1\} \times \mathbb{Z}/m$, with layer 0 outermost and $P(i,k) = r_i\, e(\theta_i + 2\pi k/m)$, $e(\varphi) = (\cos\varphi, \sin\varphi)$, and rotate so that $\theta_0 = 0$ (Definition 5.8). Let $u = \pi/m$ for odd m and $u = 2\pi/m$ for even m, and $\psi_i = \theta_i \bmod u$.

Variables. (i) One *orientation variable* per rotation orbit of 3-subsets of Λ, meaning orient > 0 for a fixed representative (7,634 orbits at $(6, 11)$, 7,820 at $(7, 10)$; Lemma 5.14); (ii) *phase-order variables* $Y(i,j) = [\psi_i < \psi_j]$, $1 \le i < j \le t-1$; (iii) for odd m, $H(i) = [\theta_i < \pi/m]$, $1 \le i \le t-1$; (iv) auxiliary *chain variables*.

Clause families.

- *phase order*: the Y variables form a strict total order (Lemma 5.19);
- *angular*: for each triple orbit and each pattern of the phase order (and H bits) of its layers, the circular order of the three directions and their antipodes is determined; where rule A or rule B applies, the orientation is fixed by a unit clause or by a clause conditional on the pattern (Lemma 5.18);
- *nesting*: $\mathrm{orient}(P(i,0), P(i,1), P(j,k)) > 0$ for $i < j$ (Lemma 5.20);
- *chains*: no chain of $K-1$ points around a pivot $(i, 0)$ as in Lemma 3.2(d) (Lemma 5.21). By rotation this covers every convex K-gon: take its outermost layer as i (Remark 5.22; not needed for soundness).

Even m needs no H variables, since antipodes are orbit points; a triple containing an antipodal pair of one layer gets no angular clause, which is weaker but sound (Section 5.8).

Two encoders. *Encoder A* is `C7UPPER/gen.py`; it classifies one sampled phase vector per pattern in binary64 arithmetic and builds the chains forwards. *Encoder B* is `C7UPPER/gen_b.py`, extended to even m as `C6UPPER/encb/gen_b2.py`; it decides circular orders by exact integer comparisons, builds the chains backwards and numbers the variables differently, and `gen_b2.py` reproduces the certified sevenfold formula of `gen_b.py` byte for byte (exact computation). The two encoders share the written soundness argument, the canonical triple representatives and part of the validation code, so they are separate implementations of one

encoding, not independent encodings. Section 5 treats encoder B as the primary object and encoder A as a cross-check.

Symmetry breaking. Besides $\theta_0 = 0$, a reflection in the x-axis maps a configuration to another one with the same layers and reversed phase order (Lemma 5.23), so the unit $Y(1,2)$ (even m) or $H(1)$ (odd m) loses no generality. Only encoder A's certificates use such a unit: $H(1)$ at $(7,8,10)$ and $Y(1,2)$ at $(6,8,11)$. Encoder B's formula is certified unsatisfiable on its own at both instances, at $(6,8,11)$ through a split into the 64 sign patterns of $Y(1,2),\dots,Y(3,4)$ (Section 6.2); so the bounds via encoder B do not use Lemma 5.23 (Theorem 5.24(a′)).

Instance sizes (exact computation). At $(6,8,11)$ both encoders write 78,059 variables and 2,387,083 clauses; by encoder A's metadata (`C6UPPER/logs/regen.log`) 10,087 of them are angular (2,347 units and 7,740 conditional) and 2,376,426 are chain clauses, and the remaining 570 are the $240 = 2\binom{10}{3}$ phase-order clauses and the $330 = 6\binom{11}{2}$ nesting units. At $(7,8,10)$ both write 80,080 variables and 2,748,538 clauses; 73,784 are angular (`C7UPPER/README.md`), $315 = 7\binom{10}{2}$ are nesting units and $168 = 2\binom{9}{3}$ phase-order clauses, which leaves 2,674,271 chain clauses (by subtraction).

Validation (evidence only unless stated otherwise).

- *Regeneration* (exact computation). Fresh runs of the generators reproduce the certified base formulas byte for byte: the sixfold base formulas of both encoders and the three sevenfold formulas (A, A + $H(1)$, B) (hashes in Appendix A; `C6UPPER/logs/regen.log`, `C6C7REVIEW/REPLAY.md`, `K8EXACT20261106`). All 64 sixfold cube files were regenerated on 6 November with `C6UPPER/tools/mkcubes.py`, and before any solving the hash of each was found equal to its record in `C6UPPER/SHA256SUMS.txt` (`C6UPPER/replay-2026-11-06`); the 6 sampled on 4 November had already been regenerated with matching hashes (`C6C7REVIEW/replay.json`). The cube sets used at solve time were checked structurally against the base-formula hashes by `verify_cover.py` (`C6UPPER/logs/verify_cover.log`, `encb_verify_cover.log`), and the regenerated ones by `verify_cover.py` (4 and 6 November) and `strict_cover.py` (4 November) (Section 6). The 32 cube files of encoder B with $\neg Y(1,2)$ have no earlier record: they were written on 6 November from the regenerated base formula (hash equal to the record), their hashes were recorded before any solving (`C6UPPER/noty12-2026-11-06/cubes.sha256`), and `verify_cover64.py` (before solving) and `verify_final64.py` (after it, from the files alone) checked byte for byte that each is the base formula with adjusted header plus six units, and that with encoder B's 32 cubes with $Y(1,2)$ they form 64 distinct sign patterns (exact computation).
- *Real sets* (exact computation). The eight sevenfold 64-point files of Section 7 and their mirror images, with the centre removed, give 16 configurations of 9 nested heptagons; with the orientation and phase variables computed exactly in $\mathbb{Z}[\zeta_7]$ and the chain variables at their least model, each satisfies all 1,765,351 clauses of both encoders' 9-layer formulas, and exactly one of each set and its mirror satisfies encoder A's formula with the unit $H(1)$ (`K8EXACT20261106`; a separate clause checker confirms the dumped assignments). The sixfold 54-point set `ES8SYM/sets/c6_54pts_9layers.pts` and its mirror satisfy every clause of both encoders' 9-layer formulas, with $Y(1,2)$ toggling (`C6C7REVIEW`); the 60-point sets A and B of Section 7 satisfy encoder A's 10-layer formula and every cut family produced on 5 November (`SIX60`).
- *Cross-encoder* (exact computation, through a shared literal-translation harness, so not independent): the non-chain parts of A and B force the same orientation literals on all 185,549 ($m = 7$) and 11,738 + 23,852 ($m = 6$, 9 and 11 layers) orbit–pattern pairs.
- Chain clauses agree with brute force over all 8-subsets on random 2- and 3-layer configurations (880 for encoder B at $m = 6$, no unsound or inexact case), and no non-chain clause is violated by random larger configurations.
- *Calibration*: for $(m,K) = (3,4),(4,5),(5,6),(6,7)$ the encoding is satisfiable exactly at the known maximum number of layers and unsatisfiable one layer beyond (the four unsatisfiable formulas were regenerated with matching hashes, solved again and certified by `cake_lpr` on 5 November, `LRATCHECK20261105/recheck`).

5 Soundness of the encodings for the $k = 8$ rotational upper bounds

The sixfold and sevenfold upper bounds for $k = 8$ rest on unsatisfiability certificates for CNF formulas written by two programs [30]:

- *encoder B*, `C6UPPER/encb/gen_b2.py` (SHA-256 `3d47459a...`), which decides the angular configuration of every triple by integer comparisons; at $(m, K, t) = (7, 8, 10)$ it reproduces byte for byte the certified encoder-B formula of `C7UPPER` (SHA-256 `b4d29017...`, written by `C7UPPER/gen_b.py`; exact computation, `C6UPPER` §3b);
- *encoder A*, `C7UPPER/gen.py` (SHA-256 `4564e7f0...`), which classifies one representative phase vector per pattern in binary64 arithmetic.

This section proves that the formulas are *sound*: every real configuration that the upper bounds exclude yields a satisfying assignment. Encoder B is the primary object; encoder A is treated as a cross-check (Proposition 5.25). The instances are $(m, K, t) = (6, 8, 11)$ and $(7, 8, 10)$; the lemmas are stated for $m \geq 3$ and $4 \leq K \leq 2m$ where this costs nothing, except that the soundness of encoder A is proved only for $m \in \{6, 7\}$. The certificates themselves, and which of their cake_lpr verdicts have been replayed against hashed files, are described in Section 6; here we prove only the implication from unsatisfiability to geometry.

Rigor labels. *Lean-checked* items are in `LAYERLEAN/LayerLemma.lean` (SHA-256 `afb8542d...`, the file as committed in `393619a57`; namespace `ErdosSzekeres.LayerLemma`; Lean 4.24.0, Mathlib `f897ebcf`; the axiom check `LAYERLEAN/logs/axioms.log` reports only `propext`, `Classical.choice`, `Quot.sound` for every cited theorem of that file except the one-line `hullVerts_subset`, which is used in `layers_are_orbits` and `exists_layer` and so is covered by their checks); the declaration names below were checked against that file, and Lean was not rerun for this section.[1] Items whose Lean names begin with `EncB` are in `ENCBLEAN`, a separate package in two parts: names beginning with `EncB.Geo` are in `ENCBLEAN/geo` (Lean 4.24.0, Mathlib `f897ebcf`), the others in `ENCBLEAN/EncB` (Lean 4.24.0 with its standard library only, no Mathlib); the axiom checks report only `propext`, `Classical.choice`, `Quot.sound` for each of them (`ENCBLEAN/logs/m9_d_recheck.log`; for `ENCBLEAN/geo`, pinned by `#guard_msgs` in `ENCBLEAN/geo/ES8Geo/Audit.lean`, `ENCBLEAN/logs/m8_fix_audit.log`). They concern DIMACS clause lists in general (Lemma 5.27), a Lean definition of encoder B's formula, not `gen_b2.py` itself, and, for the names in `EncB.Geo`, sixfold configurations. No result of this section depends on them. For $m = 6$ they form a second route from the certificates of encoder B to `Sixfold66` and the sixfold bound, Lean-checked end to end and independent of the written proofs of this section, assuming the 64 cake_lpr verdicts and the trust items (E1)–(E6) listed under gap (G2) (see also (G1)). *Written proof (not refereed)* means a complete proof given here. Most of them expand arguments reviewed on 4 November 2026 with no blocking gap found (`C6C7REVIEW/SOUNDNESS.md`). The others have had no internal review: those marked *new here* are new arguments, and those marked *not in the review* are standard facts or assembly steps that the review does not contain. Line numbers refer to `gen_b2.py` with the SHA-256 above. Statements about what a program emits are established by reading its source (Section 5.10, gap (G1)).

5.1 Configurations and layers

Let O be the origin, $e(\varphi) = (\cos\varphi, \sin\varphi)$, and ρ the counterclockwise rotation by $2\pi/m$ about O (the Lean `rotAbout` with centre O and angle $2\pi/m$). As in Section 2, $\operatorname{orient}(p, q, r) = (x_q - x_p)(y_r - y_p) - (y_q - y_p)(x_r - x_p)$ (the Lean `orient`). It is invariant under translations and rotations, changes sign under reflections, and $\operatorname{orient}(p_{\pi(1)}, p_{\pi(2)}, p_{\pi(3)}) = \operatorname{sgn}(\pi)\operatorname{orient}(p_1, p_2, p_3)$ for every permutation π. A finite set X is in *general position* if no three of its points are collinear (Lean `GeneralPosition`). It is in *convex position* if it is convex independent: Lean `ConvexPosition` is defined as Mathlib's `ConvexIndependent` of the inclusion of X, and by Mathlib's `convexIndependent_set_iff_notMem_convexHull_diff` this holds if and only if $x \notin \operatorname{conv}(X \setminus \{x\})$ for every $x \in X$. A set S is *K-free* if no K-element subset of S is in convex position, that is, if $\neg$ `HasConvexNGon` $K\ S$. For a finite set X let $\operatorname{hv}(X) = \{x \in X : x \notin \operatorname{conv}(X \setminus \{x\})\}$ (Lean `hullVerts`), and let $\operatorname{peel}(X, 0) = X$, $\operatorname{peel}(X, n+1) = \operatorname{peel}(X, n) \setminus \operatorname{hv}(\operatorname{peel}(X, n))$ (Lean `peel`).

[1] Note added in this paper: for this paper the file was checked again with `lake env lean` on 6 November 2026; see Section 3, "Build and axioms".

Definition 5.1 (nested regular m-gons)**.** A *set of t nested regular m-gons* is a set $S = L_0 \cup \cdots \cup L_{t-1}$, where each $L_i = \{\rho^k x_i : k \in \mathbb{Z}\}$ is the orbit of a point $x_i \neq O$ (a regular m-gon centred at O with m vertices), the L_i are pairwise disjoint, and $L_j \subseteq \operatorname{conv}(L_i)$ for $i < j$. Write $r_i = |x_i|$. General position and K-freeness are separate hypotheses.

Lemma 5.2 (layer lemma; Lean-checked (`layers_are_orbits`, `card_peel`, `hullVerts_eq_orbitFS_rotAbout`); the assembly below is a written step, not in the review)**.** *Let $m \geq 1$, $K \leq 2m$ and $t \geq 1$, and let S be a finite ρ-invariant K-free set with $O \notin S$ and $|S| = mt$. Put $L_n = \operatorname{hv}(\operatorname{peel}(S, n))$ for $n < t$. Then each L_n is one ρ-orbit of exactly m points, the L_n are pairwise disjoint,* $\operatorname{peel}(S, n) = L_n \cup \cdots \cup L_{t-1}$ *(in particular $S = L_0 \cup \cdots \cup L_{t-1}$), and $L_j \subseteq \operatorname{conv}(L_i)$ for $i < j$. So S, with these layers, is a set of t nested regular m-gons.*

Proof. For every n with $\operatorname{peel}(S, n) \neq \emptyset$, `layers_are_orbits` gives a point $x \in \operatorname{peel}(S, n)$ (so $x \neq O$) with $\operatorname{hv}(\operatorname{peel}(S, n)) = \{\rho^k x : 0 \leq k < m\}$, a set of exactly m points, and $\operatorname{peel}(S, n+1) \subseteq \operatorname{conv} \operatorname{hv}(\operatorname{peel}(S, n))$; as $\rho^m = \mathrm{id}$, this set is the orbit $\{\rho^k x : k \in \mathbb{Z}\}$. Since $m(t-1) < |S|$, `card_peel` gives $|\operatorname{peel}(S, n)| = |S| - mn$ for $n \leq t$; so $\operatorname{peel}(S, n) \neq \emptyset$ for $n < t$ and $\operatorname{peel}(S, t) = \emptyset$. By definition, $\operatorname{peel}(S, n)$ is the disjoint union of L_n and $\operatorname{peel}(S, n+1)$, since $\operatorname{hv}(X) \subseteq X$. Downward induction from $\operatorname{peel}(S, t) = \emptyset$ gives $\operatorname{peel}(S, n) = L_n \sqcup \cdots \sqcup L_{t-1}$, a disjoint union. For $i < j$, $L_j \subseteq \operatorname{peel}(S, j) \subseteq \operatorname{peel}(S, i+1) \subseteq \operatorname{conv}(L_i)$. □

Definition 5.3 (admissible sets)**.** Let $m \geq 3$, $4 \leq K \leq 2m$ and $t \geq 1$. A finite set $S \subseteq \mathbb{R}^2$ is (m, K, t)-*admissible* if it is ρ-invariant, $O \notin S$, $|S| = mt$, and S is in general position and K-free. Its *layers* are $L_n = \operatorname{hv}(\operatorname{peel}(S, n))$, $0 \leq n < t$. By Lemma 5.2, S with these layers is a set of t nested regular m-gons, so Definition 5.8 below applies to it.

From here on, every configuration is an admissible set with these layers. The Lean theorems are stated for $\operatorname{hv}(\operatorname{peel}(S, n))$ and $\operatorname{peel}(S, n)$, so they apply to the layers with no translation step.

Lemma 5.4 (affine images; written proof (not refereed), routine, not in the review)**.** *Let g be an affine bijection of $\mathbb{R}^2$. Then* $\operatorname{conv}(gY) = g \operatorname{conv}(Y)$ *for every $Y \subseteq \mathbb{R}^2$, so* $\operatorname{hv}(gX) = g \operatorname{hv}(X)$ *and* $\operatorname{peel}(gX, n) = g \operatorname{peel}(X, n)$ *for every finite X and every n; and g preserves general position, convex position and K-freeness. If moreover $g(O) = O$ and $g\rho g^{-1} \in \{\rho, \rho^{-1}\}$, then g maps every (m, K, t)-admissible set S to an (m, K, t)-admissible set whose layers are $g(L_0), \ldots, g(L_{t-1})$, in this order.*

Proof. g and g^{-1} map convex combinations to convex combinations, so $g \operatorname{conv}(Y) = \operatorname{conv}(gY)$. Hence, for $x \in X$, $x \in \operatorname{conv}(X \setminus \{x\})$ if and only if $gx \in \operatorname{conv}(gX \setminus \{gx\})$; this gives $\operatorname{hv}(gX) = g \operatorname{hv}(X)$, and, since the injective map g commutes with set differences, $\operatorname{peel}(gX, n) = g \operatorname{peel}(X, n)$ by induction on n. The same equivalence, applied to subsets, shows that g preserves convex position (in the form $x \notin \operatorname{conv}(X \setminus \{x\})$ for all x), hence K-freeness; g and g^{-1} preserve collinearity, hence general position. If $g\rho g^{-1} = \rho^{\pm 1}$, then also $g^{-1}\rho g = \rho^{\pm 1}$, so $\rho(gS) = g\rho^{\pm 1}(S) = gS$; here $\rho^{-1}S = S$ because ρ maps the finite set S injectively into itself. Also $O = g(O) \notin gS$ and $|gS| = |S|$. The layers of gS are $\operatorname{hv}(\operatorname{peel}(gS, n)) = g \operatorname{hv}(\operatorname{peel}(S, n)) = g(L_n)$. □

The case $g\rho g^{-1} = \rho^{-1}$ is needed only for the reflection τ of Lemma 5.23, that is, only for parts (a) and (b) of Theorem 5.24 (formulas with the reflection unit); the bounds via encoder B use part (a′) and do not need it. Elsewhere g is a rotation about O, which commutes with ρ, or, in Corollary 5.28, a translation, for which only the first part of the lemma is used.

Remark 5.5 (nested m-gons are admissible; written proof (not refereed), new here). Let $m \geq 3$ and $4 \leq K \leq 2m$. If S is a set of t nested regular m-gons $L_0, \ldots, L_{t-1}$ (Definition 5.1) that is in general position and K-free, then S is (m, K, t)-admissible and $\operatorname{hv}(\operatorname{peel}(S, n)) = L_n$ for $n < t$: its layers in the sense of Definition 5.3 are the given ones, in the given order. This is used only to restate Theorem 5.24 for sets presented as nested m-gons. It is not used for Corollary 5.28, so it is not part of the trust base of the upper bounds.

Proof. A nontrivial rotation about O fixes only O, and $x_i \neq O$, so each orbit L_i has exactly m points and avoids O; the L_i are ρ-orbits and pairwise disjoint, so S is ρ-invariant, $O \notin S$ and $|S| = mt$. Let $U_n = L_n \cup \cdots \cup L_{t-1}$; we show $\operatorname{hv}(U_n) = L_n$. Then $\operatorname{peel}(S, n) = U_n$ follows by induction on n, because $U_n \setminus L_n = U_{n+1}$. If $y \in L_j$ with $j > n$, then $y \in \operatorname{conv}(L_n) \subseteq \operatorname{conv}(U_n \setminus \{y\})$, so $y \notin \operatorname{hv}(U_n)$. Let $x \in L_n$ and $f(y) = \langle y, x \rangle$. For $y \in L_n \setminus \{x\}$ we have $f(y) = r_n^2 \cos\gamma < r_n^2 = f(x)$, where $\gamma \not\equiv 0 \pmod{2\pi}$ is the angle between x and y. Hence on $\operatorname{conv}(L_n)$ the value $f(x)$ is attained only at x (a convex combination with value

$f(x)$ puts all its weight on x). Every $y \in U_n \setminus \{x\}$ lies in $\mathrm{conv}(L_n)$ and differs from x, so $f(y) < f(x)$; the maximum of the linear function f on the polytope $\mathrm{conv}(U_n \setminus \{x\})$ is attained at a point of $U_n \setminus \{x\}$, hence is $< f(x)$, and $x \notin \mathrm{conv}(U_n \setminus \{x\})$. □

By Lemma 5.2 and Remark 5.5, the (m, K, t)-admissible sets are exactly the K-free sets of t nested regular m-gons in general position, with the same layers. Only the first direction (Lemma 5.2) enters the proofs below.

Lemma 5.6 (centre exclusion; Lean-checked (`centre_not_mem_of_even`)). *If m is even and S is ρ-invariant, in general position, and contains a point $p \neq O$, then $O \notin S$.*

Proof. $\rho^{m/2}$ is the half-turn, so $-p \in S$, and $p, O, -p$ are three distinct collinear points. □

Hence a sixfold symmetric set in general position other than $\{O\}$ has $6t$ points, and 61 points force $t \geq 11$; a sevenfold one has $7t$ or $7t+1$ points, and 65 points force $t \geq 10$. The reduction of the two upper bounds to the propositions `Sixfold66` and `Sevenfold70` (every centre-free set of 66, resp. 70, points in general position with sixfold, resp. sevenfold, rotational symmetry has a convex octagon) is Lean-checked: `convexNGon_of_layers`, `sixfold_octagon`, `sevenfold_octagon`. In the language of Definition 5.3, and after translating the centre to O, `Sixfold66` says that no $(6, 8, 11)$-admissible set exists, and `Sevenfold70` that no $(7, 8, 10)$-admissible set exists.

Lemma 5.7 (strict nesting and radius order; Lean-checked (`strict_nesting`, `dist2_le_of_later`)). *Let S be (m, K, t)-admissible with layers L_n. For $i < j$, $x \in L_i$ and $y \in L_j$ we have* $\mathrm{orient}(x, \rho x, y) > 0$*; for $i \leq j$ we have $r_j \leq r_i$.*

Proof. $x \in \mathrm{hv}(\mathrm{peel}(S, i))$ by the definition of L_i, and $y \in L_j \subseteq \mathrm{peel}(S, j)$ (Lean `hullVerts_subset`). Together with ρ-invariance, $O \notin S$, general position, K-freeness, $m \geq 2$ and $K \leq 2m$, these are the hypotheses of `strict_nesting` (layers $n = i < n' = j$) and of `dist2_le_of_later` ($n = i \leq n' = j$, which gives $|y|^2 \leq |x|^2$), both with centre O. □

5.2 Phases, genericity and the intended assignment

Definition 5.8 (phases, labels, genericity). Let S be a set of t nested regular m-gons. The directions of the points of L_i form a coset of $(2\pi/m)\mathbb{Z}$ modulo 2π, which meets $[0, 2\pi/m)$ in exactly one value θ_i, the *phase* of L_i. For $(i, k) \in \Lambda = \{0, \ldots, t-1\} \times \mathbb{Z}/m$ put $P(i, k) = r_i\, e(\theta_i + 2\pi k/m)$; then $(i, k) \mapsto P(i, k)$ is a bijection $\Lambda \to S$ and $\rho P(i, k) = P(i, k+1)$. We order Λ lexicographically, with $k \in \{0, \ldots, m-1\}$. The configuration is *normalised* if $\theta_0 = 0$. Let $u = \pi/m$ for odd m and $u = 2\pi/m$ for even m, and $\psi_i = \theta_i \bmod u \in [0, u)$; for odd m let $h_i \in \{0, 1\}$ with $\theta_i = \psi_i + h_i \pi/m$. The configuration is *generic* if $\theta_i - \theta_j \notin u\mathbb{Z}$ for all $i \neq j$. For a normalised configuration this says that $\psi_1, \ldots, \psi_{t-1}$ are pairwise distinct and nonzero ($\psi_0 = 0$); then $\theta_i \in (0, 2\pi/m)$ for $i \geq 1$, and $\theta_i \neq \pi/m$ when m is odd.

Genericity does not depend on the order of the layers, and it is preserved by rotations about O.

Lemma 5.9 (collinearity with the centre; written proof (not refereed), new here). *A set of t nested regular m-gons is generic if and only if any two distinct points $p, q \in S$ collinear with O satisfy: m is even and $q = -p$ (so p, q lie in one layer and $q = \rho^{m/2} p$).*

Proof. $P(i, k)$ and $P(j, l)$ are collinear with O if and only if their directions differ by a multiple of π, that is, $\theta_i - \theta_j + 2\pi(k-l)/m \in \pi\mathbb{Z}$. Now $\pi\mathbb{Z} + (2\pi/m)\mathbb{Z} = u\mathbb{Z}$: for odd m because $\pi = m \cdot \frac{\pi}{m}$, $\frac{2\pi}{m} = 2 \cdot \frac{\pi}{m}$ and $\gcd(m, 2) = 1$; for even m because $\pi = \frac{m}{2} \cdot \frac{2\pi}{m}$. So for $i \neq j$ some $P(i, k)$, $P(j, l)$ are collinear with O if and only if $\theta_i - \theta_j \in u\mathbb{Z}$. For $i = j$ the condition is $2(k-l) \in m\mathbb{Z}$: for odd m this forces $k = l$, and for even m it gives $k = l$ or $l = k + m/2$, i.e. $P(i, l) = -P(i, k)$. □

Lemma 5.10 (convex position is decided by orientations; written proof (not refereed), standard, not in the review). *Let X be finite and in general position, and $x \in X$. Then $x \in \mathrm{conv}(X \setminus \{x\})$ if and only if there are distinct $a, b, c \in X \setminus \{x\}$ such that* $\mathrm{orient}(b, c, x)$, $\mathrm{orient}(c, a, x)$ *and* $\mathrm{orient}(a, b, x)$ *all have the sign of* $\mathrm{orient}(a, b, c)$*. Consequently, if a bijection between two finite sets in general position preserves the sign of* orient *on all triples, it maps subsets in convex position to subsets in convex position.*

Proof. By Carathéodory's theorem in the plane, $x \in \operatorname{conv}(Y)$ implies $x \in \operatorname{conv}(T)$ for some $T \subseteq Y$ with $|T| \le 3$. If $|T| \le 2$, then x equals a point of T (impossible, $x \notin Y$) or lies on a segment between two points of X, contradicting general position. So $x \in \operatorname{conv}\{a,b,c\}$ with $\operatorname{orient}(a,b,c) \neq 0$. Write $x = \lambda_a a + \lambda_b b + \lambda_c c$ with $\lambda_a + \lambda_b + \lambda_c = 1$. Since $\operatorname{orient}(b,c,\cdot)$ is affine and vanishes at b, c, $\operatorname{orient}(b,c,x) = \lambda_a \operatorname{orient}(b,c,a) = \lambda_a \operatorname{orient}(a,b,c)$; likewise $\operatorname{orient}(c,a,x) = \lambda_b \operatorname{orient}(a,b,c)$ and $\operatorname{orient}(a,b,x) = \lambda_c \operatorname{orient}(a,b,c)$. Thus $x \in \operatorname{conv}\{a,b,c\}$ iff all $\lambda \ge 0$, and by general position no λ vanishes; this is the stated sign condition. Conversely, if the sign condition holds for distinct $a, b, c \in X \setminus \{x\}$, then $\operatorname{orient}(a,b,c) \neq 0$, so x has barycentric coordinates $\lambda_a, \lambda_b, \lambda_c$ with respect to a, b, c; the same identities show that they are positive, so $x \in \operatorname{conv}\{a,b,c\} \subseteq \operatorname{conv}(X \setminus \{x\})$. For the consequence, let $f : X \to X'$ be the bijection and $Y \subseteq X$. Subsets of a set in general position are in general position, so by the first part and the characterisation of convex position recalled in Section 5.1, Y is in convex position if and only if no $y \in Y$ and distinct $a, b, c \in Y \setminus \{y\}$ satisfy the sign condition. That condition refers only to signs of orient on triples of Y, which f preserves; so Y is in convex position if and only if $f(Y)$ is. □

Lemma 5.11 (genericity; written proof (not refereed))**.** *Let $m \ge 3$, $4 \le K \le 2m$.*

(a) If m is even, every set of t nested regular m-gons in general position is generic.

(b) If m is odd and there is an (m,K,t)-admissible set, then there is a generic, normalised (m,K,t)-admissible set (with its layers as in Definition 5.3).

Proof. (a) If $\theta_i - \theta_j \in u\mathbb{Z}$ with $i \neq j$, the proof of Lemma 5.9 gives k, l with $P(i,k)$, $P(j,l)$ collinear with O; then $P(i,k)$, $P(j,l)$ and $P(j, l + m/2) = -P(j,l)$ are three distinct points of S on one line (the first is not in L_j), contradicting general position. This is the mechanism of Lemma 5.6; no perturbation is needed.

(b) Let S be (m,K,t)-admissible, with layers L_i, radii r_i and phases θ_i (Lemma 5.2, Definition 5.8). For $\delta = (\delta_1, \dots, \delta_{t-1}) \in \mathbb{R}^{t-1}$ and $\delta_0 = 0$ put $P_\delta(i,k) = r_i\, e(\theta_i + \delta_i + 2\pi k/m)$ and $S_\delta = \{P_\delta(\lambda) : \lambda \in \Lambda\}$. For distinct labels λ, μ, ν the function $\delta \mapsto \operatorname{orient}(P_\delta(\lambda), P_\delta(\mu), P_\delta(\nu))$ is continuous and is nonzero at $\delta = 0$, by general position of S. There are finitely many label triples, so there is $\varepsilon > 0$ such that for $\|\delta\|_\infty < \varepsilon$ all these orientations have their signs at $\delta = 0$. Fix such a δ. Then: (1) distinct labels give distinct points, since $P_\delta(\lambda) = P_\delta(\mu)$ would make $\operatorname{orient}(P_\delta(\lambda), P_\delta(\mu), P_\delta(\nu)) = 0$ for any third label ν; so $|S_\delta| = mt$; (2) S_δ is in general position; (3) S_δ is ρ-invariant and $O \notin S_\delta$ (each L_i is rotated as a whole and $r_i > 0$); (4) by Lemma 5.10 applied to $P(\lambda) \mapsto P_\delta(\lambda)$ and its inverse, S_δ is K-free. So S_δ is (m,K,t)-admissible, and by Lemma 5.2 its layers are ρ-orbits. Its ρ-orbits are the rotated layers $\{P_\delta(i,k) : k\}$ (the orbits of ρ on a set are determined by the set), so its layers are these orbits in some order (in fact in the original order, because the nesting signs of Lemma 5.7 are preserved, but this is not needed), and their phases are $\theta_i + \delta_i \bmod 2\pi/m$. Since genericity does not depend on the order of the layers, S_δ is generic as soon as

$$\delta \notin B = \bigcup_{0 \le i < j \le t-1} \{\delta :\ \delta_i - \delta_j \in \theta_j - \theta_i + u\mathbb{Z}\} \qquad (\delta_0 = 0).$$

For each pair $i < j$, the values $\delta_i - \delta_j$ on the box $(-\varepsilon, \varepsilon)^{t-1}$ lie in an interval of length 4ε, which meets $\theta_j - \theta_i + u\mathbb{Z}$ in finitely many points c; each set $\{\delta_i - \delta_j = c\}$ (or $\{\delta_j = -c\}$ when $i = 0$) is a proper affine hyperplane of $\mathbb{R}^{t-1}$. A finite union of proper hyperplanes has empty interior, so the box contains a $\delta \notin B$. Finally rotate S_δ about O so that its outermost layer has phase 0: by Lemma 5.4 (a rotation about O commutes with ρ) the result is admissible with the rotated layers in the same order, and since all phases change by the same constant modulo $2\pi/m$, it is still generic. (If $t = 1$ there is nothing to perturb.) □

For odd m the perturbation is needed even though S is in general position: a point and another point on the opposite ray from O are collinear with O, which is not a point of S (the centre of an original sevenfold set is removed in the reduction). By Lemma 5.9, after the perturbation no two points of S are collinear with O.

Definition 5.12 (intended assignment)**.** Let S be a generic, normalised (m,K,t)-admissible set, with its layers $L_n = \operatorname{hv}(\operatorname{peel}(S,n))$. Its *intended assignment* α_S gives

- the phase-order variable $Y(i,j)$, $1 \le i < j \le t-1$, the value $[\psi_i < \psi_j]$;
- for odd m, the variable $H(i)$, $1 \le i \le t-1$, the value $[\theta_i < \pi/m] = [h_i = 0]$;
- the orientation variable v_c of a triple orbit with representative $c = (c_0, c_1, c_2)$ (sorted in Λ; Definition 5.13), the value $[\operatorname{orient}(P(c_0), P(c_1), P(c_2)) > 0]$;

- the chain variables the values defined in Section 5.5 (before Lemma 5.21).

In both encoders $\psi_0 = 0$ is the least phase and $h_0 = 0$. Encoder B gives layer 0 rank 0 and $h = 0$ (lines 91 and 94). In encoder A, `Enc.patterns` keeps only the orders that put layer 0 first, gives it the phase `0.0` and no H literal (`gen.py` lines 190, 211 and 218–219), and `ylit` returns the constants `True`/`False` for pairs involving layer 0 (lines 125–135); the helper `hlit` is never called. Encoder B numbers the variables $Y(i,j)$ first (lines 36–39, lexicographically in (i,j)), then $H(i)$ for odd m (line 40), then the orientation variables (lines 50–55), then the chain variables on demand (lines 161–165). Encoder A numbers the orientation variables first, then Y, then H, then the chain variables (`Enc.__init__`, `fam_chain`).

5.3 Triple orbits and orientation variables

Definition 5.13 (triple orbits, orientation variables)**.** The rotation acts on Λ by $(i,k) \mapsto (i,k+1)$ and on its 3-element subsets; the orbits of the latter are the *triple orbits*. Each encoder has one *orientation variable* v_c per triple orbit, indexed by a representative $c = (c_0, c_1, c_2)$ sorted in Λ, meant to express $\mathrm{orient}(P(c_0), P(c_1), P(c_2)) > 0$.

Encoder B's representative $\texttt{canon}(T)$ (lines 42–49) is the lexicographically least of the sorted lists $\rho^{-k}T$, $(i,k) \in T$; one variable is allocated per value of `canon` (lines 50–55). For an ordered triple of distinct labels, `olit` (lines 63–76) computes $c = \texttt{canon}(\{a,b,d\})$, takes the first point (i_0, k_0) of the list $[a,b,d]$ such that the sorted list of $\rho^{-k_0}[a,b,d]$ equals c, puts $rt = \rho^{-k_0}[a,b,d]$ and lets ω be the permutation with $c_j = rt_{\omega(j)}$; it returns v_c if $\operatorname{sgn}\omega = +1$ and $\neg v_c$ otherwise.

Lemma 5.14 (one variable per triple orbit; written proof (not refereed))**.** *(a)* `canon` *is constant on each triple orbit, and* $\texttt{canon}(T)$ *lies in the orbit of* T*; so encoder B's orientation variables correspond bijectively to the triple orbits.*

(b) Under α_S*, the literal* $\texttt{olit}(a,b,d)$ *is true if and only if* $\mathrm{orient}(P(a), P(b), P(d)) > 0$*, for every ordered triple of distinct labels.*

(c) If ρ^s *with* $s \not\equiv 0 \pmod m$ *maps a 3-set* T *onto itself, then* $3 \mid m$*,* $T = \{x, \rho^{m/3}x, \rho^{2m/3}x\}$ *lies in one layer, and* ρ^s *permutes* T *cyclically. Consequently the sign computed in (b) does not depend on which admissible rotation is used.*

For $(m,t) = (6,11)$ *there are* 45,760 *triples: the* 22 *triples* $\{(i,k),(i,k+2),(i,k+4)\}$ *form* 11 *orbits of size* 2 *and the others* 7,623 *orbits of size* 6*, in total* 7,634 *orbits. For* $(7,10)$ *all* 7,820 *orbits of the* 54,740 *triples have size* 7 *(exact computation,* `WRITEUP/note/k8_soundness_checks.py`*; the totals agree with the review's independent count,* `C6C7REVIEW/SOUNDNESS.md`*).*

Proof. (a) The candidates for $\rho^r T$ are the lists $\rho^{-(k+r)}\rho^r T = \rho^{-k}T$, $(i,k) \in T$, i.e. the candidates for T. So $\texttt{canon}(\rho^r T) = \texttt{canon}(T)$, and $\texttt{canon}(T)$ is the sorted list of some $\rho^{-k}T$. Triples in different orbits therefore have different representatives.

(b) The loop of `olit` succeeds because `canon` is attained by one of the candidates. Let $s = -k_0$. Rotations about O preserve orient and $P(\rho^s\lambda) = \rho^s P(\lambda)$, so

$$\mathrm{orient}(P(a),P(b),P(d)) = \mathrm{orient}(P(rt_0),P(rt_1),P(rt_2)) = \operatorname{sgn}(\omega)\ \mathrm{orient}(P(c_0),P(c_1),P(c_2)),$$

because (c_0,c_1,c_2) is the permutation ω of (rt_0,rt_1,rt_2) and a permutation and its inverse have the same sign. All orientations are nonzero by general position, so $\texttt{olit}(a,b,d)$, which is v_c or $\neg v_c$ according to $\operatorname{sgn}\omega$, is true under α_S exactly when $\mathrm{orient}(P(a),P(b),P(d)) > 0$. The argument holds for every admissible choice of s.

(c) A rotation $\rho^s \neq \mathrm{id}$ fixes no point of Λ (it fixes only O), so it permutes T without fixed points, i.e. as a 3-cycle. Then ρ^{3s} fixes T pointwise, so $\rho^{3s} = \mathrm{id}$ and $m \mid 3s$ while $m \nmid s$; hence $3 \mid m$ and $s \equiv \pm m/3$. Rotations preserve layers, so $T \subseteq L_i$ for one i and $T = \{x, \rho^{m/3}x, \rho^{2m/3}x\}$. Two admissible rotations s, s' for T differ by a rotation fixing the set $\rho^s T = c$; the two lists rt then differ by the identity or a 3-cycle, both even, so $\operatorname{sgn}\omega$ is the same. For $m = 6$ these are the triangles $\{(i,k),(i,k+2),(i,k+4)\}$, $k \in \{0,1\}$, two per layer, each orbit of size $6/3 = 2$; for $m = 7$ there are none. The counts follow: $(\binom{66}{3} - 22)/6 = 7{,}623$ and $\binom{70}{3}/7 = 7{,}820$. □

Part (c) is not needed for (b); it shows that the literal map never identifies an orientation variable with its own negation, which is how the stabiliser question was raised in review. Encoder A's representative is

the least sorted list $\rho^s T$ over all $s \in \mathbb{Z}/m$, and its literal function `Enc.o` is correct by the same computation as (b). Its representatives coincide with encoder B's (exact computation over all 45,760 and 54,740 triples, 0 mismatches; `C6C7REVIEW/SOUNDNESS.md`); soundness does not use this.

5.4 Directions and the angular classification

For $v \neq 0$ let $\mathrm{dir}(v) \in [0, 2\pi)$ be its direction. Let M be a set of layer indices, each $i \in M$ with a phase $\theta_i \in [0, 2\pi/m)$, such that the values $\psi_i = \theta_i \bmod u$, $i \in M$, are pairwise distinct; for odd m define h_i by $\theta_i = \psi_i + h_i\pi/m$ as in Definition 5.8. Only directions matter below, so when no configuration is given we put $P(i,k) = e(\theta_i + 2\pi k/m)$. The two cases used are the layers of a generic configuration, with its points $P(i,k)$ (normalised or not), and, in Proposition 5.25, the layers of one triple with the phases sampled by encoder A. The *objects* of a label (i,k), $i \in M$, are the point $P(i,k)$ and its antipode $-P(i,k)$ (a direction, not necessarily a point of S). Put $N = 2m$ for odd m and $N = m$ for even m, and define the cell index $J \in \{0, \dots, N-1\}$ by

$$J(P(i,k)) = (2k + h_i) \bmod 2m, \quad J(-P(i,k)) = (J(P(i,k)) + m) \bmod 2m \qquad (m \text{ odd}),$$

$$J(P(i,k)) = k, \quad J(-P(i,k)) = (k + m/2) \bmod m \qquad (m \text{ even}).$$

A *rank* is a map $\mathrm{rk} : M \to \mathbb{Z}$ with $\mathrm{rk}(i) < \mathrm{rk}(j) \iff \psi_i < \psi_j$; the *key* of an object ω of layer $i \in M$ is $\kappa(\omega) = (J(\omega), \mathrm{rk}(i))$, ordered lexicographically. These are the keys of lines 104–111.

Lemma 5.15 (key order; written proof (not refereed))**.** *Let M, the phases θ_i ($i \in M$) and a rank be as above. For objects ω of layer $i \in M$, $\mathrm{dir}(\omega) = uJ(\omega) + \psi_i$. Hence $\mathrm{dir}(\omega) < \mathrm{dir}(\omega')$ if and only if $\kappa(\omega) < \kappa(\omega')$, and $\mathrm{dir}(\omega) = \mathrm{dir}(\omega')$ if and only if $\kappa(\omega) = \kappa(\omega')$. Two distinct objects have equal keys if and only if m is even and they are $P(i,k)$ and $-P(i, k + m/2)$.*

Proof. For odd m, $\theta_i + 2\pi k/m = \psi_i + (\pi/m)(2k + h_i)$, and reducing modulo $2\pi = 2m \cdot \pi/m$ leaves $\psi_i + (\pi/m)J$ with $J = (2k + h_i) \bmod 2m$, a value in $[0, 2\pi)$ since $\psi_i < \pi/m$; the antipode adds $\pi = m \cdot \pi/m$. For even m, $\theta_i + 2\pi k/m = \psi_i + (2\pi/m)k$ with $0 \leq k < m$, and the antipode adds $\pi = (m/2) \cdot 2\pi/m$. The map $(J, \psi) \mapsto uJ + \psi$ from $\{0, \dots, N-1\} \times [0, u)$ to $[0, 2\pi)$ is strictly increasing for the lexicographic order (if $J < J'$ then $uJ + \psi < u(J+1) \leq uJ' + \psi'$), and replacing ψ by a rank does not change the lexicographic order. Equal keys mean equal J and equal rank, i.e. (the ψ_i, $i \in M$, are distinct) the same layer. Within one layer, two points or two antipodes with equal J have equal k, and a point and its own antipode have J differing by m or $m/2$. A point $P(i,k)$ and an antipode $-P(i,l)$ with equal J satisfy, for odd m, $2(k-l) \equiv m \pmod{2m}$, impossible since m is odd; for even m, $k \equiv l + m/2$. □

We call the two orientation rules *rule A* (centre inside) and *rule B* (half-plane), as in the Lean docstrings; the letters are unrelated to the names of the encoders.

Lemma 5.16 (circular dichotomy; written proof (not refereed) for (a), Lean-checked (`orient_polar`, `orient_polar_centre_inside`, `orient_polar_halfplane`, `sin_add_lt`) for the signs in (b) and (c))**.** *Let $p_1, p_2, p_3 \neq O$ be such that the six directions $\mathrm{dir}(p_a)$ and $\mathrm{dir}(p_a) + \pi$ are pairwise distinct modulo 2π. Read the six objects counterclockwise as a cyclic word in the letters* `P` *(point) and* `A` *(antipode).*

- *(a) The word alternates, or the three letters* `P` *are cyclically consecutive (a rotation of* `PPPAAA`*); the case is determined by the word.*
- *(b) (rule A) If it alternates, then $\mathrm{orient}(p_{\sigma(1)}, p_{\sigma(2)}, p_{\sigma(3)}) > 0$, where $p_{\sigma(1)}, p_{\sigma(2)}, p_{\sigma(3)}$ are the points in counterclockwise order.*
- *(c) (rule B) If it is a rotation of* `PPPAAA` *and a, b, d are the points in counterclockwise order from the first* `P` *of the block, then the counterclockwise angles x from $\mathrm{dir}\, a$ to $\mathrm{dir}\, b$ and y from $\mathrm{dir}\, b$ to $\mathrm{dir}\, d$ satisfy $x, y > 0$ and $x + y < \pi$; if moreover $|b| \geq |a|$ and $|b| \geq |d|$, then $\mathrm{orient}(a, b, d) > 0$.*

Proof. (a) Let $\alpha_1, \alpha_2, \alpha_3$ be the directions of the points in counterclockwise order, and $g_1, g_2, g_3 > 0$ the counterclockwise gaps (g_a from α_a to α_{a+1}, indices modulo 3), so $g_1 + g_2 + g_3 = 2\pi$. No gap equals π, since then a point and the antipode of another would coincide. *Case 1: all $g_a < \pi$.* Then $g_a < \pi < g_a + g_{a+1}$ (because $g_{a+2} < \pi$), so $\alpha_a + \pi$ lies in the open arc from α_{a+1} to α_{a+2}. Each of the three open arcs between consecutive points thus contains the antipode of the opposite point, and there are only three antipodes: the

word alternates. *Case 2: some gap exceeds* π; at most one can, and after a cyclic relabelling it is g_3, so $g_1 + g_2 < \pi$. Every α_a lies on the closed arc of length $g_1 + g_2$ starting at α_1, so every antipode lies on the closed arc of length $g_1 + g_2$ starting at $\alpha_1 + \pi$. That arc lies inside the open arc from $\alpha_3 = \alpha_1 + g_1 + g_2$ to $\alpha_1 + 2\pi$, because $\alpha_1 + g_1 + g_2 < \alpha_1 + \pi$ and $\alpha_1 + \pi + g_1 + g_2 < \alpha_1 + 2\pi$. So the word is `PPPAAA` up to rotation, the block begins at α_1, and $x = g_1$, $y = g_2$. The two cases give different words.

(b) Case 1. Choose representatives $\alpha < \beta < \delta < \alpha + 2\pi$ of the three directions in counterclockwise order. Then $\beta - \alpha$, $\delta - \beta$ and $\alpha - \delta + 2\pi$ are the gaps, all in $(0, \pi)$, and `orient_polar_centre_inside` (with the polar form `orient_polar`) gives orient > 0; cyclic rotation of the arguments does not change orient.

(c) Case 2 with $\alpha = \alpha_1$, $\beta = \alpha + x$, $\delta = \alpha + x + y$: `orient_polar_halfplane` needs $|a| \le |b|$, $|d| \le |b|$, $\beta - \alpha > 0$, $\delta - \beta > 0$ and $\delta - \alpha < \pi$; its proof uses $\sin x + \sin y > \sin(x + y)$ (`sin_add_lt`), as

$$\operatorname{orient}(a, b, d) = |a||b| \sin x + |b||d| \sin y - |d||a| \sin(x + y) \;\ge\; |a||d|\big(\sin x + \sin y - \sin(x + y)\big) > 0. \qquad \square$$

The case in which the six directions are not distinct is excluded by hypothesis; for a generic configuration it occurs exactly for a same-layer antipodal pair with even m (Lemma 5.15). The written dichotomy in the docstring of `gen.py` omitted this third case; it is treated in Section 5.8.

Patterns. Let c be an orbit representative with layer set M_c and free layers $F_c = M_c \setminus \{0\}$. A *pattern* $\pi = (\mathrm{rk}, h)$ consists of a bijection $\mathrm{rk} : F_c \to \{1, \dots, |F_c|\}$, extended by $\mathrm{rk}(0) = 0$, and for odd m a map $h : F_c \to \{0, 1\}$, extended by $h(0) = 0$. Encoder B enumerates all patterns (lines 89–94) and forms the premise list $\mathrm{prem}(\pi)$ (lines 96–102): for $a < b$ in F_c the literal $Y(a, b)$ if $\mathrm{rk}(a) < \mathrm{rk}(b)$ and $\neg Y(a, b)$ otherwise, and for odd m and $L \in F_c$ the literal $H(L)$ if $h(L) = 0$ and $\neg H(L)$ otherwise. The *real pattern* π_S has $\mathrm{rk}_S(L) = 1 + \#\{L' \in F_c : \psi_{L'} < \psi_L\}$ and $h_S(L) = h_L$.

Lemma 5.17 (premises; written proof (not refereed))**.** *All literals of* $\mathrm{prem}(\pi)$ *are true under* α_S *if and only if* $\pi = \pi_S$*. Moreover* rk_S *is a rank on* M_c *in the sense above.*

Proof. The ψ_L, $L \in F_c$, are distinct and positive (genericity), so rk_S is a bijection onto $\{1, \dots, |F_c|\}$ and is strictly increasing in ψ also with $\mathrm{rk}_S(0) = 0$, $\psi_0 = 0$. Under α_S, the pair literal for $a < b$ is true iff $[\mathrm{rk}(a) < \mathrm{rk}(b)] = [\psi_a < \psi_b]$, and the H literal iff $h(L) = h_L$. All pairwise comparisons determine a bijection onto $\{1, \dots, |F_c|\}$ uniquely. $\square$

Lemma 5.18 (angular clauses of encoder B; written proof (not refereed))**.** *Every clause emitted at lines 82–141 of* `gen_b2.py` *is satisfied by* α_S*.*

Proof. Fix an orbit representative c and its variable $v = v_c$. For each pattern π the code computes a value $\ell(\pi)$, either an orientation literal or `None`, and then (lines 134–141) emits the unit clause $[\ell]$ if all patterns give the same literal $\ell \neq$`None`, and otherwise the clause $\neg\mathrm{prem}(\pi) \vee \ell(\pi)$ for every π with $\ell(\pi) \neq$`None`. We show that $\ell(\pi_S)$ is `None` or true under α_S. This suffices: π_S is among the enumerated patterns, so a unit clause equals $[\ell(\pi_S)]$; and a clause for $\pi \neq \pi_S$ contains the negation of a premise literal that is false under α_S (Lemma 5.17).

For $\pi = \pi_S$ the keys of lines 104–111 are the keys κ of the six objects of the labels in c, with the rank rk_S (Lemma 5.17). If two keys coincide (line 113), then m is even and c contains a same-layer antipodal pair (Lemma 5.15), and $\ell(\pi_S) =$`None`. Otherwise the six directions are distinct, and sorting the keys (line 118) lists the objects by increasing direction from direction 0 (Lemma 5.15); the string `kinds` is the cyclic word of Lemma 5.16.

- If `kinds` alternates (line 121), `seq` $= (\sigma(1), \sigma(2), \sigma(3))$ lists the points counterclockwise, and $\ell = v$ if $\operatorname{sgn}\sigma = +1$, else $\neg v$. By Lemma 5.16(b),

  $$\operatorname{orient}(P(c_0), P(c_1), P(c_2)) = \operatorname{sgn}(\sigma)\, \operatorname{orient}(P(c_{\sigma(1)}), P(c_{\sigma(2)}), P(c_{\sigma(3)}))$$

  has the sign $\operatorname{sgn}\sigma$, so ℓ is true under α_S.
- Otherwise the word is a rotation of `PPPAAA` (Lemma 5.16(a)). In the doubled word the substring `PPP` occurs exactly where the block starts, so `dbl.find('PPP')` (line 127) is the position of its first `P`, and (a, b, d) (lines 129–130) are the points in counterclockwise order from there. If $\mathrm{layer}(b) \le \mathrm{layer}(a), \mathrm{layer}(d)$ (line 131), Lemma 5.7 gives $r_{\mathrm{layer}(b)} \ge r_{\mathrm{layer}(a)}, r_{\mathrm{layer}(d)}$, Lemma 5.16(c) gives $\operatorname{orient}(P(a), P(b), P(d)) > 0$, and $\ell =$`olit`(a, b, d) is true by Lemma 5.14(b). If the condition fails, $\ell =$`None`.

The assertions at lines 114, 117 and 128 are not used; they held when the formulas were generated. □

Exhaustively over all orbits and patterns, the six integer keys are not distinct in 622 cases (all for $m = 6$), alternate in 7,901 ($m = 6$) and 56,630 ($m = 7$) cases, form a `PPPAAA` block in 23,681 and 167,997 cases, and never do anything else (exact computation, `k8_soundness_checks.py`; a check of Lemma 5.16(a), not part of the proof). For $m = 6$, $t = 11$ encoder B emits 2,347 angular units and 7,740 conditional angular clauses (`C6UPPER` §3b).

5.5 Phase-order, nesting and chain clauses

Lemma 5.19 (phase order; written proof (not refereed))**.** *The clauses $\neg Y(i,j) \vee \neg Y(j,l) \vee Y(i,l)$ and $Y(i,j) \vee Y(j,l) \vee \neg Y(i,l)$, $1 \le i < j < l \le t-1$ (lines 79–81), are satisfied by α_S.*

Proof. The ψ_i, $i \ge 1$, are distinct reals, so $\neg Y(i,j)$ means $\psi_j < \psi_i$. The first clause is transitivity of $<$. If the second failed, then $\psi_j < \psi_i$, $\psi_l < \psi_j$ and $\psi_i < \psi_l$, a cycle. □

These clauses say that the Y variables form a strict total order; they are not used in the proofs of the other families, which only evaluate α_S.

Lemma 5.20 (nesting units; Lean-checked (`strict_nesting`) for the geometry, written proof (not refereed) for the code)**.** *The unit clauses $\texttt{olit}((i,0),(i,1),(j,k))$, $0 \le i < j \le t-1$, $k \in \mathbb{Z}/m$ (lines 143–150), are satisfied by α_S.*

Proof. $P(i,1) = \rho P(i,0)$ and $P(j,k) \in L_j$, so Lemma 5.7 gives $\mathrm{orient}(P(i,0), P(i,1), P(j,k)) > 0$; apply Lemma 5.14(b). The check at lines 148–149 only confirms that no nesting unit contradicts an angular unit. □

Chains. Fix a layer i, the pivot $p = (i,0)$ and $Q_i = \{(j,k) : j \ge i\} \setminus \{p\}$ (line 159). A *p-chain of length $l \ge 2$* is a sequence $q_1, \dots, q_l$ in Q_i with $\mathrm{orient}(P(p), P(q_s), P(q_{s+1})) > 0$ for $1 \le s < l$ and $\mathrm{orient}(P(q_s), P(q_{s+1}), P(q_{s+2})) > 0$ for $1 \le s \le l-2$. Set $\alpha_S(F_l^{(i)}(a,b)) = $ [there is a p-chain of length l with $q_1 = a$, $q_2 = b$]. With $L = K-1$, lines 166–190 emit, for the pairs (a,b) whose literal $\texttt{olit}(p,a,b)$ is not the negation of an emitted unit (`okpair`):

- (base) $F_2(a,b) \vee \neg\texttt{olit}(p,a,b)$;
- (extension) $\neg F_l(b,d) \vee \neg\texttt{olit}(p,a,b) \vee \neg\texttt{olit}(a,b,d) \vee F_{l+1}(a,b)$ for $2 \le l \le K-3$;
- (closing) $\neg F_{K-2}(b,d) \vee \neg\texttt{olit}(p,a,b) \vee \neg\texttt{olit}(a,b,d)$;

omitting the extension and closing clauses with $a \in \{b,d\}$ or with a turn literal $\texttt{olit}(a,b,d)$ that is the negation of an emitted unit. The closing clauses forbid a p-chain of length $K-1$, which together with p would be a convex K-gon.

Lemma 5.21 (chain clauses; Lean-checked (`chain_clauses_sound`) for the geometry, written proof (not refereed) for the code)**.** *Let $m \ge 3$ and $4 \le K \le 2m$. The chain clauses of lines 154–190 are satisfied by α_S.*

Proof. Base clause: if $\mathrm{orient}(P(p), P(a), P(b)) > 0$, then (a,b) is a p-chain of length 2. Extension clause: if $(b,d,q_3,\dots,q_l)$ is a p-chain and $\mathrm{orient}(P(p),P(a),P(b)) > 0$, $\mathrm{orient}(P(a),P(b),P(d)) > 0$, then $(a,b,d,q_3,\dots,q_l)$ is a p-chain of length $l+1$. (Literals are evaluated by Lemma 5.14(b).) Closing clause: otherwise there is a p-chain $(a,b,d,\dots)$ of length $K-1$, i.e. points $q_0,\dots,q_{K-2}$ of $\mathrm{peel}(S,i) \setminus \{P(p)\}$ with $\mathrm{orient}(P(p),q_s,q_{s+1}) > 0$ and $\mathrm{orient}(q_s,q_{s+1},q_{s+2}) > 0$. Here $P(p) \in L_i = \mathrm{hv}(\mathrm{peel}(S,i))$ by the definition of the layers; each q_s is $P(\lambda)$ for a label $\lambda \in Q_i$ of some layer $j \ge i$, so $q_s \in L_j \subseteq \mathrm{peel}(S,i)$ (Lemma 5.2), and $q_s \ne P(p)$ because $\lambda \ne p$ and P is a bijection. `chain_clauses_sound` (hypotheses $m \ge 3$, $4 \le K \le 2m$, $O \notin S$, ρ-invariance, general position, K-freeness) says that no such chain exists. Its proof: by `lin_pos_of_peel` all points of $\mathrm{peel}(S,i)$ other than $P(p)$ lie strictly on the inner side of the tangent to the circle of radius r_i at $P(p)$; by `hasConvexNGon_of_fan` a pivot and $n \ge 3$ chain points in an open half-plane through the pivot satisfy orient > 0 for every triple of $(x_0,\dots,x_n) = (P(p),q_0,\dots,q_{n-1})$ taken in sequence order, i.e. $\mathrm{orient}(x_s,x_{s'},x_{s''}) > 0$ for all $s < s' < s''$ (`fan_positive`; identities `orient_trans_halfplane`, `fan_step`) and are therefore in convex position (`ErdosSzekeres.Suk.convexPosition_of_positive_triples`, in `ErdosSzekeres/Suk/OrientationClique.lean`, covered by the axiom check of `hasConvexNGon_of_fan`); this gives K points in convex position. Omitted clauses only weaken the formula. □

Remark 5.22 (completeness; written proof (not refereed), not used). Conversely, if S has K points X in convex position, the closing clauses fail under α_S. Let i be the least layer met by X and $P(i,k) \in X$; then $\rho^{-k}X$ is in convex position, contains $P(p)$, and its other points have labels in Q_i. Listing $\rho^{-k}X$ counterclockwise along its convex hull from $P(p)$ gives $P(p), q_1, \dots, q_{K-1}$, and every three vertices of a convex polygon in general position, taken in counterclockwise order, are positively oriented. So $(q_1, \dots, q_{K-1})$ is a p-chain of length $K-1$. Thus the chain family expresses K-freeness exactly; soundness needs only Lemma 5.21.

Encoder A's chains. `fam_chain` of `gen.py` builds the chains forwards: $R_l(q,q')$ means a p-chain of length l ending with q,q', and it is assigned that truth value. The base, extension and closing clauses are justified as above by appending instead of prepending. Encoder A also uses the emitted angular and nesting units (recorded in `forced`) in two ways. If the unit $[\ell]$ is emitted, it drops the literal $\neg\ell$ from a chain clause; since α_S satisfies $[\ell]$ (Proposition 5.25, for $m \in \{6,7\}$), it satisfies the shortened clause exactly when it satisfies the full one. If the unit $[\neg\ell]$ is emitted, it omits every chain clause containing $\neg\ell$, as encoder B does; such a clause is subsumed by the unit, and omitting it is a weakening.

5.6 The reflection unit

Lemma 5.23 (reflection; written proof (not refereed))**.** *Let S be a generic, normalised (m,K,t)-admissible set, and $\tau(x,y) = (x,-y)$. Then τS is again a generic, normalised (m,K,t)-admissible set, its layers are $\tau(L_0), \dots, \tau(L_{t-1})$ in this order, and its phases are $\theta'_0 = 0$ and, for $i \geq 1$, $\theta'_i = 2\pi/m - \theta_i$, with $\psi'_i = u - \psi_i$ and, for odd m, $h'_i = 1 - h_i$. Hence $\alpha_{\tau S}(Y(i,j)) = \neg\alpha_S(Y(i,j))$ for $1 \leq i < j$, and for odd m $\alpha_{\tau S}(H(i)) = \neg\alpha_S(H(i))$. In particular exactly one of S and τS satisfies the unit $Y(1,2)$ (even m, $t \geq 3$), resp. $H(1)$ (odd m, $t \geq 2$).*

Proof. τ is a linear bijection with $\tau(O) = O$ and $\tau\rho\tau^{-1} = \rho^{-1}$. By Lemma 5.4, τS is (m,K,t)-admissible and its layers are $\tau(L_0), \dots, \tau(L_{t-1})$, in this order; $\tau(L_i)$ is the ρ-orbit of τx_i, a regular m-gon of radius r_i. Since $\tau(r\,e(\varphi)) = r\,e(-\varphi)$, the directions of $\tau(L_i)$ are $-\theta_i + (2\pi/m)\mathbb{Z}$; the phase is 0 for $i = 0$ and $2\pi/m - \theta_i$ for $i \geq 1$, because $\theta_i \in (0, 2\pi/m)$. Differences of phases change sign modulo $2\pi/m \in u\mathbb{Z}$, so genericity is kept. For even m, $\psi'_i = \theta'_i = u - \psi_i$. For odd m, $\theta_i = \psi_i + h_i\pi/m$ with $\psi_i \in (0, \pi/m)$ gives $\theta'_i = (\pi/m - \psi_i) + (1-h_i)\pi/m$ with $\pi/m - \psi_i \in (0, \pi/m)$, so $\psi'_i = \pi/m - \psi_i$ and $h'_i = 1 - h_i$. The map $\psi \mapsto u - \psi$ reverses the order of distinct phases. □

The reflection need not be a symmetry of S: it produces another configuration of the same kind. Layer indices must not be permuted, since they encode the radial order used by rule B and by the nesting and chain clauses; the reflection keeps them.

The reflection unit is used only by encoder A, the cross-check. Encoder B's formulas are certified unsatisfiable on their own: at $(7,8,10)$ directly, and at $(6,8,11)$ through the 64 cubes of Section 5.9. For them Theorem 5.24(a′) suffices, and its proof does not use Lemma 5.23.

5.7 The soundness theorem

Theorem 5.24 (soundness; written proof (not refereed), resting on the Lean-checked lemmas cited above)**.** *Let $m \geq 3$, $4 \leq K \leq 2m$ and $t \geq 3$. Let F_B and F_A be the CNF formulas written by* `gen_b2.py` *and* `gen.py` *on input (m,K,t), and let the* reflection unit *be the unit clause $Y(1,2)$ for even m and $H(1)$ for odd m, in each encoder's own numbering. Suppose that some (m,K,t)-admissible set exists, that is, a set of mt points in general position, invariant under ρ, not containing O, with no K points in convex position. By Remark 5.5 this holds in particular if some set of t nested regular m-gons in general position has no K points in convex position. Then:*

(a) F_B plus the reflection unit is satisfiable;
(a′) F_B is satisfiable, by an argument that does not use Lemma 5.23;
(b) if moreover $m \in \{6,7\}$, then F_A plus the reflection unit is satisfiable.

Part (b) is restricted to $m \in \{6,7\}$ because encoder A classifies triples in binary64 arithmetic, and the rounding bound used in Proposition 5.25(i) has been computed only for these two values of m (it does not

depend on K or t). For other m the same argument needs that bound for the given m; we make no claim about F_A there. Encoder B uses only integer comparisons, so parts (a) and (a′) need no such bound.

Part (a′) follows from (a), since a model of F_B plus a unit clause is a model of F_B. Its own proof below omits the reflection step, so that the bounds via encoder B (Corollary 5.28) use neither Lemma 5.23 nor the case $g\rho g^{-1} = \rho^{-1}$ of Lemma 5.4. Part (a′) was added on 6 November 2026, after $F_B(6, 8, 11)$ had been certified unsatisfiable on its own (Section 5.9).

For the certified instances: at $(6, 8, 11)$ both formulas have 78,059 variables and 2,387,083 clauses (SHA-256 `479e31a9...` for B, `542c8a3a...` for A), and $Y(1, 2)$ is variable 1 of F_B (lines 36–39) and variable $7{,}635 = 7{,}634 + 1$ of F_A; at $(7, 8, 10)$ both have 80,080 variables and 2,748,538 clauses (`b4d29017...` for B, `1a7c0e70...` for A), and $H(1)$ is variable $\binom{9}{2} + 1 = 37$ of F_B and $7{,}820 + 36 + 1 = 7{,}857$ of F_A. The sevenfold certificate for A is for F_A plus $H(1)$ (`fa297b5b...`, made by `C7UPPER/addunits.py`); the one for B is for F_B alone, the case of part (a′). At $(6, 8, 11)$ the certificate for B is also for F_B alone, through 64 cubes, and the one for A is for F_A plus $Y(1, 2)$, through 32 cubes (Section 5.9). Sizes and hashes: `C6UPPER/README.md`, `C7UPPER/README.md`, `C7UPPER/SHA256SUMS.txt`.

Proof. Let S be an (m, K, t)-admissible set. If m is odd, replace it by the generic, normalised admissible set of Lemma 5.11(b). If m is even, S with its layers is a set of nested regular m-gons (Lemma 5.2) in general position, hence generic by Lemma 5.11(a); rotate it about O so that it is normalised, which keeps it admissible with the rotated layers in the same order (Lemma 5.4) and keeps it generic. (a′) Give the variables of F_B their values under α_S (Definition 5.12); every variable of F_B is a Y, H, orientation or chain variable. By reading the source, F_B consists of the header (line 192) and exactly the clauses appended to `cls` at lines 80–81, 137, 141, 147, 175 and 190, that is, the families of Lemmas 5.19, 5.18, 5.20 and 5.21; all are satisfied. Neither Lemma 5.23 nor the reflection τ is used. (a) By Lemma 5.23, τS is again a generic, normalised (m, K, t)-admissible set, and exactly one of α_S and $\alpha_{\tau S}$ satisfies the reflection unit; replace S by τS if necessary. Then α_S satisfies the reflection unit, and by the argument of (a′) it satisfies F_B. (b) Let $m \in \{6, 7\}$, take S as in (a), so that α_S satisfies the reflection unit, and give the variables of F_A their values under α_S. `build` calls `fam_order`, `fam_angle`, `fam_nest` and `fam_chain` and nothing else (the Grassmann–Plücker family of the docstring is not emitted by the program). The first three are satisfied by Lemma 5.19 (identical clauses) and Proposition 5.25, the last by the paragraph on encoder A's chains. □

Proposition 5.25 (encoder A's angular and nesting clauses; written proof (not refereed), new here, and exact computation)**.** *Let $m \in \{6, 7\}$, $4 \le K \le 2m$, $t \ge 3$, and let S be as in Definition 5.12. Every clause emitted by* `fam_angle` *and* `fam_nest` *of* `gen.py` *on input (m, K, t) is satisfied by α_S. Two independent justifications:*

(i) (written proof (not refereed), with the rounding bound by exact computation; new here) the argument below, for $m \in \{6, 7\}$ and every K and t;

(ii) (exact computation, one program sharing `gen.py`*'s literal translation) only for the two certified instances,* `C6UPPER/encb/compare_ab2.py` *and* `C7UPPER/tests/compare_ab.py` *compared, for every triple orbit having a clause in either encoder and every pattern of its layers, the set of orientation literals forced by A's angular and nesting clauses with that of B's clauses without chain variables: at $(6, 8, 11)$* 10,417 *clauses each,* 5,297 *orbits,* 23,852 *orbit–pattern pairs,* 0 *mismatches (*`C6UPPER/logs/encb_compare.log`*); at $(7, 8, 10)$* 74,099 *clauses each,* 6,221 *orbits,* 185,549 *pairs,* 0 *mismatches (*`C7UPPER/logs/repro_validation.log`*). With Lemmas 5.18 and 5.20, an A clause whose premises are true under α_S forces a literal that a B clause with true premises also forces, hence a true one; an A clause with a false premise is satisfied.*

Proof of (i). `fam_nest` emits the same nesting units as encoder B (Lemma 5.20). In `fam_angle`, a pattern is a permutation `perm` of M_c with 0 first, plus H bits for odd m; its premises are the literals $[\psi_{\mathrm{perm}(r)} < \psi_{\mathrm{perm}(r+1)}]$ for consecutive entries not involving layer 0 (`ylit` makes those constant) and the H literals. Since $\psi_0 = 0$ is the least phase, all premises are true under α_S iff `perm` is the ψ-order of M_c and the bits are the h_L; the emission rule is the same as in Lemma 5.18. For the pattern of S, `patterns` supplies phases $\psi_L = u\, q_L/(n+1)$, $n = |M_c|$, $q_L \in \{0, \dots, n\}$ increasing along `perm`, and $\theta_L = \psi_L + (1 - [\mathrm{bit}])\pi/m$ for odd m. These realise the same ψ-order and the same h as S on M_c. Lemma 5.15 applies to both phase assignments on M_c (the sampled ψ_L are distinct and $\theta_L \in [0, 2\pi/m)$), and they give the six objects of c the same keys. So the objects have the same cyclic word, the same counterclockwise order of points and the same block as in S. `angle_config` sorts the three directions, forms the gaps, returns `degenerate` if a gap is within 10^{-9} of 0 or π, `in` with the sign

of the sorting permutation if all gaps are $< \pi$, and otherwise the points in order starting after the largest gap; by Lemma 5.16 these are exactly B's three cases and outputs, *provided* the floating-point decisions agree with the exact ones. They do: every exact direction is an integer multiple of $\delta_0 = u/(n+1)$, and so is π ($\pi = m(n+1)\delta_0$ for odd m, $\frac{m}{2}(n+1)\delta_0$ for even m); every exact direction is at most $2\pi - \delta_0$; and the three points have distinct directions (distinct ψ for distinct layers, distinct k in one layer). So each exact gap equals π or lies at distance at least $\delta_0 \geq \pi/(4m)$ from both 0 and π (exactly, the least such distance is $\pi/12$ for $(m,t) = (6,11)$ and $\pi/28$ for $(7,10)$). The float inputs depend only on m, on $n \in \{1,2,3\}$, on q_L, on the bit and on k, not on K or t. Re-evaluating `gen.py`'s float expressions in all 54 ($m = 6$) and 105 ($m = 7$) such cases and comparing with the exact angles (π enclosed by 36-digit rationals) shows two things (exact computation, `k8_soundness_checks.py`). First, each value `theta[i] + 2*math.pi*k/m` that `angle_config` reduces is a float in $[0, 2\pi)$, so the reduction `% (2*math.pi)` returns it unchanged (a wrap-around would have produced an error near 2π). Second, the value used is within $8.7 \cdot 10^{-16}$ of the exact direction. Hence the sorted order is the exact one, each computed gap is within 10^{-14} of the exact gap (two angle errors, the error of `2*math.pi`, and at most two roundings of numbers below 8, each below $9 \cdot 10^{-16}$), gaps equal to π (only same-layer antipodal pairs, even m) are reported as `degenerate`, and all other comparisons with `eps`$= 10^{-9}$ and with π are decided correctly. The representative's classification is then that of S, and the emitted literal is true under α_S exactly as in Lemma 5.18. □

5.8 The omitted antipodal case

Lemma 5.26 (diameter; written proof (not refereed); the formula is not in the review)**.** *Let m be even, q and $-q$ points of one layer, and p another point. Then*

$$\operatorname{orient}(q, -q, p) = -2\det(q,p) = -2|q||p|\sin\big(\operatorname{dir}(p) - \operatorname{dir}(q)\big),$$

so $\operatorname{orient}(q,-q,p) > 0$ *if and only if p lies strictly clockwise from q by less than π. For a generic normalised configuration this is decided by the keys of q, $-q$ and p (Lemma 5.15), i.e. by the phase pattern alone.*

Proof. $\operatorname{orient}(q,-q,p) = \det(-q-q, p-q) = -2\det(q,p) + 2\det(q,q) = -2\det(q,p)$, and $\det(q,p) = |q||p|\sin(\operatorname{dir} p - \operatorname{dir} q)$; $\operatorname{dir}(p) \neq \operatorname{dir}(\pm q)$ by Lemma 5.9. □

A triple containing a same-layer antipodal pair has the centre on one of its sides, so it is neither a triangle around the centre nor contained in an open half-plane through the centre. For every pattern of such an orbit, encoder B finds coinciding keys (line 113) and encoder A a gap of π (`degenerate`); neither emits an angular clause. For $(m,t) = (6,11)$ these are 352 of the 7,634 orbits ($11 \cdot 3 \cdot 64$ triples in orbits of size 6; exact computation, `k8_soundness_checks.py`). A nesting triple $\{(i,0),(i,1),(j,k)\}$, $j > i$, contains no same-layer antipodal pair (as $m \geq 4$), and rotations preserve this property, so none of these orbits carries a nesting unit either; their orientations are constrained only through the chain family. By Lemma 5.26 valid clauses for them exist and are omitted. Omitting valid clauses is a weakening: it can only enlarge the set of satisfying assignments, and Theorem 5.24 concerns only the emitted clauses. So the omission is not a source of unsoundness, and the certificates prove unsatisfiability without these clauses. More generally the encoders omit every orientation fact not covered by rules A and B (for instance half-plane triples whose middle point is not on the outermost layer, and the two-point-chord facts of `CHORD20261105`); each omission is a weakening. The chord-lemma family of `CHORD20261105` (written proof (not refereed), each sixfold clause checked by exact computation) contains such diameter clauses; it was not part of the `C6UPPER` or `C7UPPER` formulas. The `gen.py` docstring states the angular dichotomy without this third case (finding of `C6C7REVIEW/SOUNDNESS.md`); the emitted formulas are unaffected.

5.9 The cube cover and the passage to geometry

Lemma 5.27 (cube cover; written proof (not refereed); Lean-checked for DIMACS clause lists)**.** *Let F be a CNF formula and $x_1, \ldots, x_n$ variables. If $F \wedge \ell_1 \wedge \cdots \wedge \ell_n$ is unsatisfiable for each of the 2^n choices $\ell_j \in \{x_j, \neg x_j\}$, then F is unsatisfiable.*

Proof. A model β of F satisfies the cube with $\ell_j = x_j$ if $\beta(x_j)$ is true and $\ell_j = \neg x_j$ otherwise. □

For formulas given as lists of DIMACS clauses the lemma is the composition of two Lean-checked theorems, `EncB.not_dimacsSatisfiable_of_cover` applied to `EncB.covers_signPatterns` (`ENCBLEAN/EncB/Cover.lean`). No geometric meaning of the x_j is needed, so sign patterns of the Y variables that contradict transitivity need no separate argument (their cubes were refuted anyway). Two sixfold splits are used.

The split of $F_B(6,8,11)$ (encoder B, 64 cubes). Here F is encoder B's base formula itself, and $x_1,\dots,x_6$ are $Y(1,2), Y(1,3), Y(1,4), Y(2,3), Y(2,4), Y(3,4)$: variables $1,2,3,10,11,18$ (lines 36–39); 40 of the 64 patterns contradict transitivity. The 32 cubes with $Y(1,2)$ are encoder B's cubes of the split below; the 32 with $\neg Y(1,2)$ were written on 6 November 2026 in the same layout with the first unit negated. That each of the 64 cube files is the base formula byte for byte, with adjusted header, plus six unit clauses on these variables, and that the 64 patterns are distinct, was checked by `verify_cover64.py` and, from the files alone, by `verify_final64.py` (exact computation; `C6UPPER/noty12-2026-11-06`). By Lemma 5.27, unsatisfiability of the 64 cube formulas gives that $F_B(6,8,11)$ is unsatisfiable, the first hypothesis of Corollary 5.28(a). For this split no variable needs a geometric meaning. For a Lean definition of $F_B(6,8,11)$ this application of the lemma is Lean-checked: `EncB.encB_unsat_of_cubes64`, where `EncB.cubes64` lists the 64 sign patterns of the variables $1,2,3,10,11,18$ in the order of the cube files. That the printed texts of that definition and of its 64 cubes are the certified files is checked by hashes only (gap (G2)).

The split of the base formula plus $Y(1,2)$ (both encoders, 32 cubes each). Here F is the base formula plus the unit $Y(1,2)$, and $x_1,\dots,x_5$ are $Y(1,3), Y(1,4), Y(2,3), Y(2,4), Y(3,4)$: variables $2,3,10,11,18$ in encoder B and 7,636, 7,637, 7,644, 7,645, 7,652 in encoder A (by the variable numbering described in the paragraph after Definition 5.12); 20 of the 32 patterns contradict transitivity. That each of the 32 cube files of each encoder is the base formula byte for byte, with adjusted header, plus $Y(1,2)$ and the five literals of its pattern, and that the 32 patterns are distinct, was checked by `C6UPPER/tools/verify_cover.py` and by the assertion-independent `C6C7REVIEW/strict_cover.py` (exact computation). Only the variable of the unit $Y(1,2)$ must have its geometric meaning; it is fixed by the source (Theorem 5.24). This is the route of encoder A, the cross-check; for encoder B these 32 cubes are one half of the split above.

Corollary 5.28 (from unsatisfiability to the Lean propositions; written proof (not refereed), not in the review, with the cited Lean-checked reduction)**.** *Let F_A, F_B be as in Theorem 5.24.*

(a) If $F_B(6,8,11)$ is unsatisfiable, then `Sixfold66` *holds; with* `sixfold_octagon` *(Lean-checked), every sixfold rotationally symmetric set of at least* 61 *points in general position contains a convex octagon. The same conclusion follows if, for one encoder, the* 32 *formulas "base $(6,8,11)$ + $Y(1,2)$ + cube" are all unsatisfiable (this route uses Lemma 5.23, through Theorem 5.24(a), resp. (b); it is the route of encoder A, the cross-check).*

(b) If $F_A(7,8,10)$ plus $H(1)$, or $F_B(7,8,10)$, is unsatisfiable, then `Sevenfold70` *holds; with* `sevenfold_octagon` *(Lean-checked), every sevenfold rotationally symmetric set of at least* 65 *points in general position contains a convex octagon.*

Proof. (a) Suppose `Sixfold66` fails: some T in general position, invariant under the rotation by $2\pi/6$ about a point $c \notin T$, with $|T| = 66$, has no convex octagon. Let $\eta(x) = x - c$ be the translation and R_c the rotation in `Sixfold66` (`rotAbout` with centre c and angle $2\pi/6$), so that $R_c(x) = c + \rho(x - c)$ (with $m = 6$). Then $T - c = \eta(T)$ is $(6,8,11)$-admissible: it is in general position and 8-free by the first part of Lemma 5.4, which holds for every affine bijection (its last clause assumes $g(O) = O$ and is not used here, since $\eta(O) = -c$); it is ρ-invariant because $\eta R_c \eta^{-1}(y) = R_c(y + c) - c = \rho(y)$, so $\rho(\eta T) = \eta R_c(T) = \eta T$ (here $R_c T = T$, as R_c maps the finite set T injectively into itself); $O \notin \eta(T)$ because $c \notin T$; and $|\eta(T)| = 66$. By Theorem 5.24(a′), $F_B(6,8,11)$ is satisfiable, which contradicts the first hypothesis; this route does not use Lemma 5.23. Under the second hypothesis, by Theorem 5.24(a), resp. (b), the base formula of the encoder plus $Y(1,2)$ is satisfiable; by Lemma 5.27, so is one of the 32 cube formulas, a contradiction. (b) Replace $2\pi/6$, 66 and $(6,8,11)$ by $2\pi/7$, 70 and $(7,8,10)$ in the first part of the argument; then $T - c$ is $(7,8,10)$-admissible. No cube step is needed: unsatisfiability of F_A plus $H(1)$ contradicts Theorem 5.24(b), and unsatisfiability of F_B contradicts Theorem 5.24(a′). □

For the Lean definition `EncB.encB` of F_B (gap (G1)), the first statement of (a) is Lean-checked without the written proofs of this section: `EncB.Geo.sixfold66_of_unsat` derives `Sixfold66` from the unsatisfiability of `EncB.encB` at $(6,8,11)$, through a polar labelling of the peeled layers (`EncB.Geo.labelling_of_counterexample`), the model of the formula that it defines (`EncB.Geo.modelOfLabelling`)

and `EncB.encB_satisfiable`. With the cube cover, the reading of the printed texts and `sixfold_octagon` it is part of `EncB.Geo.sixfold_es8_of_texts` (gap (G2)).

The hypotheses of the corollary are supplied by certificates (cake_lpr, `s VERIFIED UNSAT`). Their status, including how many verdicts are bound to file hashes by a replay against freshly generated formulas, is stated in Section 6 and is not repeated here. The link from the certified files to the programs analysed here is that fresh runs reproduce the formula files byte for byte[2] (hashes above; `C6UPPER/logs/regen.log`, `C6C7REVIEW/REPLAY.md`).

5.10 Trust base and gaps

Ingredient	Status
Strict nesting, radius order, centre exclusion (Lemmas 5.6, 5.7); reduction to `Sixfold66`, `Sevenfold70`	Lean-checked (`LAYERLEAN`)
Layer lemma (Lemma 5.2)	Lean-checked core; the assembly of the peeled layers is a written proof (not refereed), not in the review; the part that the Lean route for encoder B (last row) uses, a polar labelling of the layers for even m, is Lean-checked
Signs of rules A and B (Lemma 5.16(b),(c)); chain soundness (Lemma 5.21, geometry)	Lean-checked
Collinearity with the centre (Lemma 5.9)	written proof (not refereed), new here
Convex position via orientations; affine images (Lemmas 5.10, 5.4; for the case $g\rho g^{-1} = \rho^{-1}$ see the last row)	written proof (not refereed), standard; not in the review
Genericity and perturbation (Lemma 5.11)	written proof (not refereed); proves in full the review's hyperplane statement
Orbit variables and stabilisers; key order; circular dichotomy (a); premises (Lemmas 5.14, 5.15, 5.16, 5.17)	written proof (not refereed)
Code-to-clause correspondence for both encoders (Lemmas 5.18–5.21, Theorem 5.24)	written proof (not refereed) by reading the source
Cube cover (Lemma 5.27; 64 cubes for $F_B(6,8,11)$, 32 for $F_A(6,8,11)$ plus $Y(1,2)$)	written proof (not refereed); Lean-checked for DIMACS clause lists (`ENCBLEAN`)
Encoder A's float classification, $m \in \{6,7\}$ only (Proposition 5.25)	exact computation (comparison with B at the two instances; rounding bound), and written proof (not refereed), new here
Cube files equal base plus units; orbit and pattern counts; formula hashes	exact computation
Omitted antipodal clauses (Lemma 5.26)	written proof (not refereed), formula not in the review; a weakening only
Passage to `Sixfold66`, `Sevenfold70` (Corollary 5.28)	written proof (not refereed), not in the review; for encoder B at $(6,8,11)$ also Lean-checked end to end, assuming the 64 cake_lpr verdicts and (E1)–(E6) (last row)
Unsatisfiability of the formulas	certificate (Section 6)
Not needed for the upper bounds: nested m-gons are admissible (Remark 5.5)	written proof (not refereed), new here
Used only by encoder A (cross-check), not for the bounds via encoder B: the reflection unit (Lemma 5.23) and the case $g\rho g^{-1} = \rho^{-1}$ of Lemma 5.4	written proof (not refereed); Lemma 5.4 not in the review
Lean route for encoder B at $m = 6$ (`ENCBLEAN`; gaps (G1), (G2)): for a Lean definition of F_B, satisfaction by the assignment of every combinatorial model (even $m \geq 2$), a model from every admissible set (even m; polar labelling of the layers), the 64-cube cover at $(6,8,11)$, the round trip of its printed text, and the assembly with `sixfold_octagon` (`EncB.Geo.sixfold_es8_of_texts`)	Lean-checked end to end (`ENCBLEAN/EncB`: standard library only; `ENCBLEAN/geo`: Mathlib `f897ebcf`), assuming the 64 cake_lpr verdicts and the trust items (E1)–(E6) of gap (G2): that the printed texts are the certified files is evidence only (compiled code, hashes), and cake_lpr's parser is tested, not verified; that the definition equals `gen_b2.py`'s output is evidence only and not used on this route; not for $m = 7$

Table 2: What the implication "certificates ⇒ `Sixfold66`, `Sevenfold70`" rests on. The last three rows are not used by the written proof for encoder B: the first of them is used by neither encoder, the second only by encoder A, and the third is a second route for encoder B at $m = 6$, Lean-checked end to end (assuming the 64 cake_lpr verdicts and the trust items (E1)–(E6)) and independent of the written proof.

[2]Note added in this paper: for the 64 sixfold cube files of the hash-bound replay of 6 November 2026 this is on record by hash for all of them, and structurally for all; the 32 cube files of encoder B with $\neg Y(1,2)$, written on 6 November, are the regenerated base formula plus six units, checked byte for byte; see Section 4, "Validation".

Gaps that remain.

(G1) The correspondence between the Python sources and the clause families is established by reading the code at the stated hashes; there is no formal semantics of the programs, and the Python interpreter is trusted to run them as written. For encoder B with $m = 6$ this gap is closed in Lean on a second route (`ENCBLEAN`), for a Lean definition of its formula, `EncB.encB`: a Lean-checked proof that for even $m \geq 2$ it is satisfied by the assignment derived from every `EncB.Model` (`EncB.encB_satisfiable`), and a Lean-checked proof that for even m every admissible set yields an `EncB.Model`, through a polar labelling of its layers (`EncB.Geo.labelling_of_counterexample`, `EncB.Geo.modelOfLabelling`; with Mathlib). An `EncB.Model` is a combinatorial interface: an orientation of each triple of labels and an order of the layer phases, together with, as hypotheses, the properties that the clause families use (the orientation is alternating and invariant under ρ, the phase order is a strict total order with layer 0 first, rules A and B hold on the circular order of the points and their antipodes, the nesting relations hold, and there is no chain of $K-1$ points around a pivot). On that route the Python source is not used: the Lean definition is tied to the certified files by the hashes of its printed texts (trust item (E1) under (G2)). That `EncB.encB` equals the output of `gen_b2.py` is not proved; it was checked by running compiled code (equal bytes on 222 of the 225 instances of a small grid, a refusal of the Lean program on the other 3, where `gen_b2.py` raises, and equal bytes on the recorded instances, among them $F_B(6, 8, 11)$). So (G1) stands for the written proof, which argues about `gen_b2.py`, and in particular for $m = 7$ and for encoder A; for $m = 6$ via encoder B the Lean route replaces it by the trust items of (G2).

(G2) The passage from a DIMACS file to `Sixfold66` or `Sevenfold70` is a written proof (Corollary 5.28). For encoder B at $(6, 8, 11)$ it is formalized end to end for the Lean definition of (G1), up to the trust items (E1)–(E6) below (`ENCBLEAN`): `EncB.Geo.sixfold_es8_of_texts` states that if each of the 64 Lean strings that the package's printer function computes for that formula with one of the 64 cubes is parsed, by a DIMACS parser written in the package, to an unsatisfiable clause list, then every set of at least 61 points in general position that is invariant under the rotation by $2\pi/6$ about some point contains a convex octagon (`EncB.Geo.sixfold_es8_order6_of_texts`: the same for the rotation by $2\pi/6$ or by $2\pi \cdot 5/6$). It composes the round trip of the printed texts (`EncB.parseDimacs_render`, `EncB.parseDimacs_cube64_get`, `EncB.cubesUnsat64_of_texts_get`), the 64-cube cover (`EncB.encB_unsat_of_cubes64`), the two Lean proofs of (G1) and `sixfold_octagon`. Its hypothesis is equivalent to the unsatisfiability of the 64 cube formulas of the Lean definition (`EncB.Geo.texts_iff_cubesUnsat64`), so a failing parse cannot make it hold trivially; it says exactly what the 64 verdicts are taken to certify. Not Lean-checked, and trusted on this route:

- (E1) that these 64 texts are the certified cube files: the SHA-256 of the output of the compiled printer equals the record for the base formula and for all 64 cube files (`ENCBLEAN/scripts/hash_binding.sh`, `ENCBLEAN/logs/m9_a2fix_hash_binding.log`); evidence from running compiled code, not a Lean theorem;
- (E2) the 64 hash-bound cake_lpr verdicts `s VERIFIED UNSAT` on these files, and the provenance of the cake_lpr binary that gave them (the proof files are not public); the records that tie each verdict to the hash of its file were written by the Python harnesses of the two runs (`C6UPPER/replay-2026-11-06/replay.py`, `C6UPPER/noty12-2026-11-06/noty12.py`) and cross-checked by `verify_final64.py`;
- (E3) that cake_lpr reads each of these files as the package's parser does, and that its `s VERIFIED UNSAT` means unsatisfiability of the clause list it read; the second half rests on cake_lpr's verified HOL4/CakeML checker, the first is not verified (tested on samples only, `ENCBLEAN/logs/m9_a2_cake_parse.log`);
- (E4) the Lean compiler and runtime, for the printing program of (E1) (not for the theorems), and a reading of that program's input/output code (that it prints exactly these texts and nothing else);
- (E5) the Lean kernel, Mathlib `f897ebcf`, and the match of the Lean definitions (`GeneralPosition`, `HasConvexNGon`, `orient`, `rotAbout`, `Invariant`) with the notions of Section 5.1;
- (E6) SHA-256.

So the sixfold bound is formally verified end to end via encoder B, assuming the 64 cake_lpr verdicts and the trust items (E1)–(E6). The results of this section other than the cited Lean theorems are written proofs only, and the sevenfold bound and the route of encoder A are not formally verified end

to end.

(G3) Encoder A's angular part rests on binary64 arithmetic. For the two certified instances Proposition 5.25 covers it twice; for other (K, t) with $m \in \{6, 7\}$ only argument (i) applies, and for other m no claim about encoder A is made. Argument (i) is new here and unreviewed, and comparison (ii) is a single program that uses `gen.py`'s own literal translation.

(G4) The written proofs marked *new here* (Lemma 5.9, Remark 5.5 and Proposition 5.25(i)) and those marked *not in the review* (Lemmas 5.10 and 5.4, the assembly in Lemma 5.2, the formula of Lemma 5.26, and Corollary 5.28) have had no internal review. The remaining written proofs expand arguments reviewed on 4 November. Part (a′) of Theorem 5.24 and the 64-cube split of Section 5.9 were added on 6 November, after that review; the proof of (a′) is that of (a) without the reflection step. Lemma 5.23 and the case $g\rho g^{-1} = \rho^{-1}$ of Lemma 5.4 are used only by encoder A (cross-check); the bounds via encoder B (Theorem 5.24(a′), Corollary 5.28) do not depend on them. The Lean route for encoder B at $m = 6$ of (G2) uses none of the written proofs of this section.

Where the sources only sketched. The docstring of `gen.py` (step 0) calls K-freeness a strict inequality in the parameters; Lemma 5.10 supplies the needed fact that convex position is decided by orientation signs. The review states that the bad phase equalities form a finite union of proper affine hyperplanes near the configuration; Lemma 5.11(b) proves this in full and identifies the perturbed set as an admissible set through Lemma 5.2. The angular dichotomy is completed by the antipodal case (Lemma 5.15 and Section 5.8), as the review asked. The chain argument of the docstring and of the review (a simple, locally convex fan polygon) is replaced by the Lean-checked proof through positive triples. Stating everything for admissible sets with their peeled layers lets the Lean theorems apply to the layers directly; the link to nested m-gons given abstractly is Remark 5.5, which the upper bounds do not need.

6 The certificates

Checkers. All formulas of the original runs were solved by CaDiCaL 3.0.1 (commit `c6073042`) with text LRAT output and checked by `cake_lpr`; the sixfold replay of 6 November used CaDiCaL 2.1.3, and so did the run of the same day for encoder B's 32 cubes with $\neg Y(1, 2)$, which were solved only then (Section 6.2). A certificate is the line `s VERIFIED UNSAT` of `cake_lpr` for a specific CNF file and LRAT file. The link from a certified file to the analysed programs is regeneration: fresh runs of the generators reproduce the base formulas byte for byte, and the cube files are the base formula plus units, which is checked structurally and, for all 64 cube files of the original split, by hash; the 32 cube files with $\neg Y(1, 2)$ were written from the regenerated base formula and checked byte for byte (Section 4, "Validation"); the link to geometry is Corollary 5.28, and for encoder B's 64 sixfold cube files also the Lean theorem `EncB.Geo.sixfold_es8_of_texts` (Section 8). For encoder B's sixfold base formula and its 64 cube files there is a second link, to Lean definitions (`ENCBLEAN`; Section 8): a compiled Lean program prints the Lean formula `EncB.encB` at $(6, 8, 11)$, alone and with each of the 64 cubes of the Lean list `EncB.cubes64`, and the SHA-256 of each printed text equals the recorded hash of the certified file, 65 of 65 (`ENCBLEAN/logs/m9_a2fix_hash_binding.log`); this is evidence only, since it rests on running compiled code, and it is the trust item (E1) of the Lean route (Section 5.10, gap (G2)). The script `ENCBLEAN/scripts/resolve.sh` re-solves any of these 64 cube formulas, printed by that program, with the two binaries of the replay, after checking the formula's hash against its records; it was run on 2 of them, and `cake_lpr` accepted both fresh proofs (`ENCBLEAN/logs/m9_resolve_fix.log`). `lrat-check` also accepted every proof of the original runs (it was not used in the replay of 6 November or in the run for $\neg Y(1, 2)$); by the defect described in Section 2 this is supporting information only. For the review replays of 4 November the binaries had SHA-256 `e12ecc8c...` (CaDiCaL), `2fee767a...` (`cake_lpr`) and `f2046212...` (`lrat-check`) (`C6C7REVIEW/REPLAY.md`); for the sixfold replay of 6 November, `4acf5860...` (CaDiCaL 2.1.3) and `c14ff498...` (`cake_lpr`) (`C6UPPER/replay-2026-11-06/TOOLS_AND_PROOFS.sha256`); the same two binaries ran for the cubes with $\neg Y(1, 2)$, checked by hash at the start and at the end of that run (`C6UPPER/noty12-2026-11-06/TOOLS_AND_PROOFS.sha256`). The replay thus used a different `cake_lpr` build from the one used in the review of 4 November; building commit `033aad53` of the public `cake_lpr` repository (`github.com/tanyongkiam/cake_lpr`) with its Makefile on the same workstation reproduces it byte for byte (`C6UPPER/replay-2026-11-06/README.md`).

6.1 Sevenfold

Encoder A's base formula at $(7, 8, 10)$ plus the unit $H(1)$ (variable 7,857, appended by `C7UPPER/addunits.py`), and encoder B's formula alone, were refuted (Table 3). On 4 November both proofs were replayed with both checkers against freshly generated formulas, with matching proof and formula hashes (`C6C7REVIEW/REPLAY.md`); on 6 November the three 10-layer formulas were regenerated again, byte for byte (`K8EXACT20261106`).

formula	header	CNF (SHA-256)	CaDiCaL	LRAT bytes	LRAT (SHA-256)	`cake_lpr`
A + $H(1)$	`80080 2748539`	`fa297b5b...`	833 s	1,702,278,368	`c750a805...`	34 s
B	`80080 2748538`	`b4d29017...`	802 s	1,630,259,870	`75f9bd35...`	32 s

Table 3: The sevenfold certificates (`C7UPPER`); "header" gives the variables and clauses of the `p cnf` line, and both `cake_lpr` runs print `s VERIFIED UNSAT`. CaDiCaL peak memory 1.6–1.7 GB; `cake_lpr` with heap 8,000 MB, about 8.4 GB peak. Full hashes in Appendix A.

6.2 Sixfold

Split. The base formula plus the unit $Y(1, 2)$ is split into the 32 sign patterns of $Y(1, 3)$, $Y(1, 4)$, $Y(2, 3)$, $Y(2, 4)$, $Y(3, 4)$ (Lemma 5.27). Each cube file is the base formula, with adjusted header, plus $Y(1, 2)$ and the five literals of its pattern; that this holds for all 32 files of each encoder and that the patterns are distinct was checked by `C6UPPER/tools/verify_cover.py` and by the separately written `C6C7REVIEW/strict_cover.py`, which also reject a missing cube, a changed clause, a flipped unit and a changed header (exact computation). Twenty patterns contradict transitivity and are refuted at once (under 1 s each); the other twelve are the orders of $\psi_1, \ldots, \psi_4$ with $\psi_1 < \psi_2$ (Table 4).

For encoder B the base formula is also split without the unit: into the 64 sign patterns of $Y(1, 2), Y(1, 3), Y(1, 4), Y(2, 3), Y(2, 4), Y(3, 4)$ (variables $1, 2, 3, 10, 11, 18$; Lemma 5.27 and Section 5.9). Its 32 cubes with $Y(1, 2)$ are the cubes just described; the other 32 carry $\neg Y(1, 2)$, and of these, twenty patterns contradict transitivity and twelve are the orders of $\psi_1, \ldots, \psi_4$ with $\psi_2 < \psi_1$ ("The cubes with $\neg Y(1, 2)$" below).

		encoder A			encoder B		
cube	order of $\psi_1, \ldots, \psi_4$	wall (s)	LRAT (GB)	LRAT SHA-256	wall (s)	LRAT (GB)	LRAT SHA-256
`c00000`	4<3<1<2	1,626	3.18	`7919b16d1ede`	2,151	3.90	`a76391d0ca6b`
`c00001`	3<4<1<2	1,741	3.34	`1c774429b8ea`	2,543	4.50	`01aeee875c67`
`c01001`	3<1<4<2	1,698	3.17	`7714cc436f9a`	1,768	3.26	`113f8bd2439b`
`c01011`	3<1<2<4	1,876	3.61	`741e807910af`	2,102	3.91	`068025c93371`
`c10000`	4<1<3<2	1,385	2.73	`19f963b53698`	1,716	3.18	`09027ddd6318`
`c10100`	4<1<2<3	2,065	3.96	`64f107981df4`	2,553	4.58	`00d233f47def`
`c11000`	1<4<3<2	1,945	3.73	`5d3abe7a99a4`†	1,990	3.62	`407e30d112a7`†
`c11001`	1<3<4<2	2,068	3.91	`2a9960bf2572`	2,055	3.74	`88dcf4f58f01`†
`c11011`	1<3<2<4	1,284	2.80	`4385dd740793`	1,916	3.60	`e75fbbd361f9`
`c11100`	1<4<2<3	1,230	2.88	`34570be0b2d2`	1,495	2.78	`513d09595a4a`
`c11110`	1<2<4<3	1,500	3.63	`16fbfe9c1c7d`†	2,076	4.20	`c968041d158d`
`c11111`	1<2<3<4	1,579	3.89	`230df1f31447`†	2,292	4.46	`0a988c3f26bb`†
total, 12 phase orders			40.8			45.7	

Table 4: The twelve sixfold phase-order cubes (`C6UPPER`). "wall": solve wall time of the runner lane (`C6UPPER/logs/cubes.txt`, `encb_cubes.txt`), not CaDiCaL's own process time, which is 3–10 s less (`cadical_summary.txt`, `encb_cadical_summary.txt`). Every proof, and every proof of the twenty contradictory cubes, was accepted by `cake_lpr` when it was produced (3 November). †: `cake_lpr` verdict replayed on 4 November against freshly generated formulas with matching hashes (`C6C7REVIEW/replay.json`). Times, sizes and hashes are those of the proofs of 3 November. In the hash-bound replay of 6 November all 24 formulas of this table were solved again; the fresh proofs, which differ from those listed, were all accepted by `cake_lpr` ("What has been replayed" below; hash prefixes in Appendix A; `C6UPPER/replay-2026-11-06/results.tsv`). Sizes from the byte counts in `C6UPPER/SHA256SUMS.txt`; all 32 proofs together take 41.0 GB (A) and 46.5 GB (B).

Runs. Encoder A: one core per cube, 8 lanes, 56 minutes wall clock, about 20,000 CPU seconds; CaDiCaL peak memory 1.3–2.0 GB per phase-order cube (`C6UPPER/logs/cadical_summary.txt`). Encoder B: 6 lanes, 1 h 24 min wall clock, about 24,700 CPU seconds, 1.5–1.7 GB. `cake_lpr` ran one proof at a time (heap 8,000 MB for A, 4,000 MB for B); the completion times in `C6UPPER/logs/cake.txt` and `C6UPPER/logs/encb_cake.txt` span 31 and 84 minutes for the 32 proofs of each encoder. An unsplit run of the whole instance was stopped after 6,578 s without an answer; it proves nothing.

What has been replayed. All 64 cube proofs (32 per encoder) were accepted by `cake_lpr` when they were produced, on 3 November (`C6UPPER/logs/cake.txt`, `C6UPPER/logs/encb_cake.txt`); those logs record cube labels, not formula or proof hashes. On 4 November a review replayed three randomly chosen phase-order proofs per encoder (marked †) with both checkers against freshly generated formulas, with matching hashes (`C6C7REVIEW/REPLAY.md`), so, before the replay below, 6 of the 64 `cake_lpr` verdicts were bound to file hashes; the same day all 64 proofs, with hashes matching the recorded ones, were replayed against freshly generated formulas with `lrat-check` only, which is not a certificate (`C6UPPER/logs/replay_2026-11-04.txt`).

In the hash-bound `cake_lpr` replay of 6 November 2026 (`C6UPPER/replay-2026-11-06`) the following was done and recorded, for each encoder:

- the base formula was regenerated, and its hash had to equal the record;
- the 32 cube files were rebuilt with `mkcubes.py`, and `verify_cover.py` reported a complete cover;
- the hash of each cube file was compared with its record in `C6UPPER/SHA256SUMS.txt` before any solving, and all 64 matched.

Each cube formula was then solved again with CaDiCaL 2.1.3, the SHA-256 of the fresh proof was recorded, and the proof was checked by `cake_lpr`. Every solver run ended UNSAT, and all 64 proofs were accepted (`s VERIFIED UNSAT`); none was rejected. Each verdict is recorded together with the hashes of its formula and proof (`results.tsv`), and an independent check of that record confirmed the hashes and verdict lines.

- The 40 fresh proofs of the contradictory cubes are byte-identical to the archived ones.
- The 24 fresh phase-order proofs differ from those of Table 4; for these formulas the certificate is the fresh proof.

The run took 63,539 s of CaDiCaL wall time, first on one lane and then on six. The accepted proofs are kept compressed (28.4 GB, outside the repository, not public).

The cubes with $\neg Y(1,2)$ (encoder B). On 6 November 2026, from 12:47 to 15:40 KST, the 32 cube formulas "encoder B base $(6,8,11) + \neg Y(1,2)$ + pattern" were written, hashed before any solving and checked as stated in Section 4 ("Validation"); then each was solved with CaDiCaL 2.1.3 (four lanes) and its proof was hashed and checked by `cake_lpr` (one at a time), with the binaries of the replay (`C6UPPER/noty12-2026-11-06`). Every solver run ended UNSAT, and `cake_lpr` accepted all 32 proofs (`s VERIFIED UNSAT`): 32 of 32, none rejected. Each verdict is recorded together with the hashes of its formula and proof (`results.tsv`). The run took 35,411 s of CaDiCaL time (0.8 to 3,921 s per cube: 2,320 to 3,921 s for each of the twelve phase-order cubes, at most 1.0 s for each of the twenty others) and 2,471 s of `cake_lpr` time. The proofs total 56.3 GB; they are kept compressed (14.9 GB of `.lrat.zst`, 16.8 GB for the whole data directory with the cube files and logs; outside the repository, not public). A check run after the job from the files alone (`verify_final64.py`, result PASS) confirmed the tool, base-formula and cube hashes, the cover of all 2^6 patterns, and, for each of the 32 new cubes, that its accepted row, the file on disk, the hash list written before solving, the `cake_lpr` log line and the decompressed proof agree; each of the other 32 cubes has the hash of an accepted encoder-B row of the replay. These 64 certificates and Lemma 5.27 give that encoder B's base formula $F_B(6,8,11)$ (SHA-256 `479e31a9...`) is unsatisfiable.

6.3 The upper bounds

Theorem 6.1 (upper bounds; certificates, two encoders, each verdict bound to formula and proof hashes: for $m = 7$ by the replay of 4 November, for $m = 6$ by the hash-bound `cake_lpr` replay of 6 November (64 of 64 cube formulas) and the run for encoder B's cubes with $\neg Y(1,2)$ of the same day (32 of 32); on top of the written soundness proof (not refereed) of Section 5 and the Lean-checked reduction of Proposition 3.3)**.**

`Sevenfold70` *and* `Sixfold66` *hold. Hence every sevenfold rotationally symmetric set of at least* 65 *points in general position, and every sixfold one of at least* 61 *points, contains* 8 *points in convex position.*

Proof. By Table 3, $F_A(7,8,10)$ plus $H(1)$ and $F_B(7,8,10)$ are unsatisfiable (certificates); by Corollary 5.28(b), `Sevenfold70` holds. By Section 6.2 ("What has been replayed" and "The cubes with $\neg Y(1,2)$"), the 64 cube formulas of encoder B's split into the 2^6 sign patterns of $Y(1,2),\dots,Y(3,4)$ are unsatisfiable (certificates), so $F_B(6,8,11)$ is unsatisfiable by Lemma 5.27; by Corollary 5.28(a), `Sixfold66` holds, and this route does not use the reflection lemma (Lemma 5.23). As a cross-check, by the replay of 6 November all 32 formulas "base $(6,8,11)$ $+Y(1,2)$ + cube" of encoder A are unsatisfiable (certificates), which gives `Sixfold66` again by the second part of Corollary 5.28(a), with Lemma 5.23. Proposition 3.3 gives the rest. One encoder suffices in each case; the second is a cross-check. □

For $m = 6$ the sixfold statement also follows from the 64 encoder-B certificates by a route that uses neither the written soundness proof nor the Python generators: the Lean theorem `EncB.Geo.sixfold_es8_of_texts` (Section 8, "Encoder B in Lean"), under the trust items (E1)–(E6) of Section 5.10.

7 The lower-bound constructions

Theorem 7.1 (exact computation)**.** *(a) The five files* `SIX60/sets/c6_60pts_X.pts`*, where X is one of A, B, C, C2 and D, describe sixfold symmetric sets of* 60 *points (ten orbits, no centre) in general position without* 8 *points in convex position, of five pairwise different order types (called A, B, C, C′, D).*

(b) The eight files of `PRIMEFOLD/sets/k8` *describe sevenfold symmetric sets of* 64 *points (the centre and nine orbits) in general position without* 8 *points in convex position, of seven pairwise different order types (*`p7_compound_64` *and* `p7_two_orbit_64` *have the same orientation table).*

Format. Each file lists one point per line as integers $(w_0,\dots,w_{m-1})$ for the point $\sum_j w_j\zeta^j$, $\zeta = e^{2\pi i/m}$. Multiplication by ζ is a cyclic shift of this vector, so the symmetry is exact. Every orbit is $\{\zeta^j s : 0 \le j < m\}$ for a *seed* s. In every file the rows other than the centre are distinct and split into orbits of exactly m rows, each consisting of the m cyclic shifts of its first row in the file, which we take as its seed; in all files except `p7_compound_64`, `p7_two_orbit_64` and `m7_x2_G9s_centre`, whose rows are listed cluster by cluster (`rule_p7_64pts` is also built from clusters but is stored by orbits), the orbits are consecutive blocks of m rows. This, and the hash of every file against `SIX60/sets/SHA256SUMS.txt` and `PRIMEFOLD/SHA256SUMS`, is checked by `WRITEUP/P1/tools/make_tables.py`. For $m = 6$ every point is $a + b\zeta$ with integers a, b, i.e. a point of the hexagonal lattice; the linear map $a + b\zeta \mapsto (a, b)$ has positive determinant, so all orientation signs are signs of integer determinants. For $m = 7$ the signs are signs of elements of $\mathbb{Z}[\zeta_7]$, decided exactly. Two examples with small coordinates, as seeds (a, b) for $a + b\zeta$:

D `SIX60/sets/c6_60pts_D.pts` (SHA-256 `e2eaceb9e6483b46...`; scale 2^{19}): $(524288, 0)$, $(131119, 11665)$, $(131103, 11664)$, $(107744, 21423)$, $(107703, 21423)$, $(88128, 30720)$, $(86762, 30677)$, $(56942, 30072)$, $(56913, 30089)$, $(5569, 389)$.

`c7_64pts_small`
(SHA-256 `ebe028f65c3856a9...`; coefficients of at most 3 digits): the centre and $(661, -486)$, $(30, 179)$, $(211, -32)$, $(220, -206)$, $(55, -179)$, $(164, -37)$, $(164, -163)$, $(42, 102)$, $(-110, -29)$.

Appendix B lists the seeds of all thirteen files, with the scale of each sixfold file and the number of digits of the largest coefficient of each sevenfold file (from 3 for `c7_64pts_small` to 31 for `m7_x2_G9s_centre`).

How the sets were found (the claims rest only on the exact checks below). *Sixfold* (`SIX60/construction`). Sets A and B were found by *alternating projections*: starting from a real 10-layer configuration with few convex octagons (90 in the best start), a SAT solver on encoder A's formula, together with all cut clauses produced earlier, was steered towards that configuration's orientation signs, real configurations were fitted to the resulting abstract models, and local annealing on the exact number of convex octagons drove it to 0; the coordinates were then made exact in $\mathbb{Z}[\zeta_6]$. Set C was found independently by a *phase-fixed LP-CEGAR* loop: with the phases fixed, every orientation is the sign of a linear form in the inverse radii, so existence of

radii for a sign vector is a linear program, and each infeasible sign vector yields a Farkas clause; after 2,284 SAT/LP rounds one perturbed phase vector gave a realizable octagon-free sign vector (LP margin $9.3 \cdot 10^{-4}$). C′ is C rounded at scale 10^9 (its own exact check gives a different order type), and D is A after a constrained local optimisation that keeps the octagon count 0 and raises the smallest normalized sine from $2.2 \cdot 10^{-5}$ to $1.3 \cdot 10^{-3}$, with coefficients at most 2^{19}. The shapes differ: A is a spiral of near-twin layers, B and C have needles of eight layers near the midpoints of the outer edges. *Sevenfold* (`PRIMEFOLD` §3, `ES8SYM` §6). Four sets come from simulated annealing of layer radii and phases with exact rounding (two annealers, each rounded at two scales); `p7_compound_64` is one orbit of a two-level 9-point cluster, `rule_p7_64pts` comes from an explicit cluster rule, and `m7_x2_G9s_centre` is one orbit of chord-aligned flat 9-point clusters, each with the centre.

Verification (exact computation). *Sixfold*: each set was checked by three separately written implementations (`SIX60/README.md`, section "Verification"; for D, which is `cand1r_s19.pts`, the pipeline run is in `SIX60/construction/REALIZE_README.md`, "cand1r"): (i) the pipeline of `ES8SYM/tools` (`verify_cfg.py`: exact algebraic signs and, separately, interval arithmetic; `cvx2.c`: two differently anchored dynamic programs for convex subsets; `cvxcheck.c`: depth-first search; `crosscheck_decimal.py`; `enum_convex.c`), run by the finders, and for C and C′ also the separate checker `indep_check.py`; (ii) `SIX60/verify/verify60.py`, standard library only, in the oblique integer coordinates: distinctness, absence of the origin, closure under $(a, b) \mapsto (-b, a+b)$, all 34,220 triples non-collinear, and the largest convex subset by two independently anchored $O(n^4)$ dynamic programs (validated against brute force on 150 random small sets and on planted convex 9-gons); (iii) `SIX60/verify/dfs_convex.c`, an exhaustive depth-first enumeration of all subsets in convex position with exact `__int128` orientations. *Sevenfold*: `PRIMEFOLD/check64` (written for an independent check) verifies distinctness, exact 7-fold invariance and all 41,664 orientation signs exactly in $\mathbb{Z}[\zeta_7]$ (and again with 110-digit decimals), and counts convex k-subsets by an anchored dynamic program and again by a separate C enumerator working on the sign table; the five files that are also in `ES8SYM/sets` were checked by the `ES8SYM` pipeline as well, and the other three on 7 November 2026 (`WRITEUP/P1/checks-20261107`); `K8EXACT20261106` recomputed all orientation signs of all eight files with a separately written $\mathbb{Z}[\zeta_7]$ sign routine and found agreement with `PRIMEFOLD/check64/z7exact.py`. So every sevenfold convex-subset count rests on four or more separately written programs. *Re-run for this draft* (6 November 2026, on copies, under the resource caps of the run): `check64/run_all.sh` on all eight sevenfold files (35 s) and `verify60.py` and `dfs_convex.c` on all five sixfold files (2.6–3.5 s each for `dfs_convex`) reproduced every hash, every collinearity count (0) and every convex-subset count of Tables 5 and 6 (`WRITEUP/P1/rerun-20261106`).

set	convex 3	convex 4	convex 5	convex 6	convex 7	convex 8
A	34,220	292,353	1,058,232	1,454,781	296,544	0
B	34,220	291,927	1,057,422	1,462,500	301,674	0
C	34,220	284,577	954,066	1,110,345	221,976	0
C′	34,220	284,607	954,234	1,111,230	223,416	0
D	34,220	297,279	1,128,000	1,651,268	316,182	0

Table 5: Numbers of k-subsets in convex position of the five sixfold 60-point sets (exact computation, `dfs_convex.c`; `SIX60/logs`).

file (`PRIMEFOLD/sets/k8`)	convex 3	convex 4	convex 5	convex 6	convex 7	convex 8	origin
`c7_64pts_small`	41,664	416,052	2,077,152	5,009,081	4,688,763	0	annealing
`c7_64pts_9layers_centre`	41,664	416,220	2,078,804	5,011,594	4,691,381	0	same, other scale
`c7_64_own_s1000`	41,664	392,966	1,770,020	3,836,413	3,328,145	0	second annealer
`c7_64pts_9layers_centre_own`	41,664	392,553	1,760,612	3,786,951	3,252,475	0	same, other scale
`p7_compound_64`*	41,664	470,764	3,142,335	11,670,659	18,396,499	0	two-level clusters
`rule_p7_64pts`	41,664	487,711	3,585,162	15,450,148	30,031,745	0	cluster rule
`m7_x2_G9s_centre`	41,664	462,581	2,962,260	10,286,640	14,719,287	0	flat clusters

Table 6: Numbers of k-subsets in convex position of the sevenfold 64-point sets (exact computation, `PRIMEFOLD/check64/check64.log`). *: `p7_two_orbit_64` has the same orientation table and counts.

Order types. The numbers of convex k-subsets do not change under relabelling or reflection. The counts of convex 4-subsets in Tables 5 and 6 are pairwise different within each table, so the sets of each part of Theorem 7.1 have pairwise different order types, also up to reflection. The classical Erdős–Szekeres 64-point set has 422,186 convex 4-subsets and no non-trivial automorphism, so it is not order-type equivalent to any set with a rotation or reflection symmetry (exact computation, `ES8SYM` §6). Clusters of blow-ups in the sense of Baek and Balko [2] are line-modules, an order-type invariant (written lemma, not refereed, `ES8SYM/proofs/structure.md`). So the following excludes order-type equivalence to such blow-ups with clusters of the stated sizes (exact computation): `c7_64pts_small` and `c7_64pts_9layers_centre` have no line-module with 2 to 60 points, and `c7_64pts_9layers_centre_own` and `c7_64_own_s1000` none with 2 to 56 (`ES8SYM` §6, `PRIMEFOLD/check64/modules60`, `WRITEUP/P1/checks-20261107`); each of these two has 22 non-trivial line-modules, of sizes 57, 60, 61 and 62. The last three sets of Table 6 are built from 9-point clusters and do have line-modules, among them seven of size 9 (`check64.log`); whether they are equivalent to blow-ups was not checked.

Consistency with the encodings. As stated in Section 4, the sevenfold sets and their mirrors, without the centre, satisfy both encoders' 9-layer formulas (exact computation, `K8EXACT20261106`), and the sixfold set A, labelled by encoder A's convention, gives an assignment of all 5,710 orbit signs and the phase-order variables that extends to a model of encoder A's 10-layer formula and satisfies all 2,157,010 clauses of the 74 cut files produced on 5 November (exact computation, `SIX60`). A real configuration satisfies every sound clause, so these are consistency checks of the encodings and of the cuts; they are not part of the proof.

Remark 7.2 (history of the sixfold value). Until 5 November the largest sixfold octagon-free set known to us had 54 points, and in our records the sixfold maximum was either 54 or 60. The plain encoding is satisfiable at 10 layers, and abstract 10-layer models satisfying all 3-term Grassmann–Plücker relations had resisted realization (evidence only, `ROT456` §3); several of them were excluded one at a time for the prescribed sixfold rotation (written proofs, not refereed, with identities checked by exact computation; `C6C7REVIEW`, `ROTREFINE20261104`). The question was settled by construction. Since set A lies in the phase-order cube $\psi_1 < \psi_2 < \psi_3 < \psi_4$, that 10-layer cube could never have become unsatisfiable.

8 Trust and rigor

Table 7 lists every ingredient of Theorem 1.1 with its label. The soundness argument has its own, finer table (Table 2) and list of gaps (G1)–(G4) in Section 5.10.

What is trusted. On the route of the written proof: the Lean 4 kernel and the Mathlib definitions used in the statements; `cake_lpr` and the machine that ran it; the Python interpreter, to run the generators as written; the programs named for each exact computation (for the cube cover with $Y(1,2)$ two separately written checkers, `verify_cover.py` and `strict_cover.py`, and for encoder B's 64-cube cover the two checks `verify_cover64.py` and `verify_final64.py`, written for that run; for the lower bounds at least three per set, Section 7); SHA-256, to identify files; and the reading of the generator sources on which the code-to-clause correspondence rests (gap (G1)). On that route the passage from a DIMACS file to `Sixfold66` or `Sevenfold70` is a written proof (gap (G2)). For the sixfold bound via encoder B there is a second route, Lean-checked end to end (next paragraph); it trusts only the 64 `cake_lpr` verdicts and the items (E1)–(E6) of gap (G2) in Section 5.10: (E1) that the certified files are the texts printed by the compiled Lean program (their SHA-256), (E2) the 64 verdicts of `cake_lpr` and the provenance of its binary, (E3) that `cake_lpr` parses these files as the Lean parser does and that its verdict means unsatisfiability of what it parsed, (E4) the Lean compiler and runtime for the printing program, and a reading of its input/output code (that it prints exactly these texts and nothing else), (E5) the Lean kernel, Mathlib `f897ebcf` and the match of the Lean definitions with the notions of Section 2, and (E6) SHA-256. The Python generators (`gen_b2.py`, `mkcubes.py`), the cube-cover checks and the written soundness proof are not on that route. The records that tie each verdict to the hash of its file were written by the Python harnesses of the two runs (`C6UPPER/replay-2026-11-06/replay.py`, `C6UPPER/noty12-2026-11-06/noty12.py`) and cross-checked by `verify_final64.py`; they belong to (E1) and (E2).

Ingredient	Status
Centre exclusion for even m; strict nesting; radius order; polar form, rules A and B; chain soundness (geometry); reduction to `Sixfold66`, `Sevenfold70` (Section 3)	Lean-checked (`LAYERLEAN`; re-checked 6 November)
Layer lemma: decomposition into nested layers (Lemmas 3.1 and 5.2)	Lean-checked core; the assembly of the peeled layers is a written proof (not refereed), not in the review (gap (G4)); on the Lean route for $m = 6$ via encoder B, the labelling of the layers that it uses is Lean-checked
Soundness of the encodings: genericity, orbit variables, key order, circular dichotomy, premises, code-to-clause correspondence, reflection unit, cube cover, passage to the Lean propositions (Section 5)	written proof (not refereed); most parts expand arguments reviewed on 4 November; parts marked *new here* or *not in the review* unreviewed (gap (G4)); the reflection unit (Lemma 5.23) is used only by encoder A (cross-check); details in Table 2; not used by the Lean route for $m = 6$ via encoder B (row "Sixfold bound via encoder B in Lean")
Encoder A's floating-point classification, $m \in \{6, 7\}$	exact computation (comparison with B, rounding bound) and written proof (not refereed), Proposition 5.25
Sixfold bound via encoder B in Lean (`ENCBLEAN`; "Encoder B in Lean" below): from the unsatisfiability of the 64 cube texts printed by a Lean program for a Lean definition of the formula to Theorem 1.1(a) for $m = 6$ and to the first sentence of Corollary 1.2 for order 6, in the stronger form with 61 points (`EncB.Geo.sixfold_es8_of_texts`, `EncB.Geo.sixfold_es8_order6_of_texts`)	Lean-checked end to end (`ENCBLEAN/EncB`: standard library only; `ENCBLEAN/geo`: Mathlib `f897ebcf`), assuming the 64 `cake_lpr` verdicts and the trust items (E1)–(E6) of Section 5.10: that the printed texts are the certified files is evidence only (compiled code; 65 of 65 hashes), and that `cake_lpr` parses them as the Lean parser does is tested, not verified; a second route, independent of the written proof; not for $m = 7$
Certified files are the output of the analysed programs	exact computation: base formulas and all 64 cube files of the original split regenerated with matching hashes (the cube files on 6 November); two structural checkers; the 32 cube files with $\neg Y(1, 2)$ written from the regenerated base formula and checked byte for byte
Cube files are the base formula plus the stated units; the 32 patterns of each encoder's split with $Y(1, 2)$, and the 64 of encoder B's split, are distinct	exact computation (two checkers for each split)
Unsatisfiability, sevenfold: A $+ H(1)$ and B	certificate (`cake_lpr`; both replayed against regenerated formulas with matching hashes)
Unsatisfiability, sixfold, encoder B: the base formula $F_B(6, 8, 11)$, through 64 cube formulas (all sign patterns of $Y(1, 2), \ldots, Y(3, 4)$)	certificate: 64 hash-bound `cake_lpr` verdicts (32 from the replay of 6 November, 32 from the run for $\neg Y(1, 2)$ of the same day, 32 of 32 accepted), each bound to formula and proof hashes; with the cube cover (Lemma 5.27)
Unsatisfiability, sixfold, encoder A (cross-check): 32 cube formulas with $Y(1, 2)$	certificate: hash-bound `cake_lpr` replay (6 November 2026; 64 of 64 accepted with encoder B's 32 cubes with $Y(1, 2)$), each verdict bound to formula and proof hashes (before it: solve-time verdicts by cube label, 6 of 64 hash-bound on 4 November)
Lower bounds: twelve order types in thirteen files	exact computation (three programs per sixfold set, four or more per sevenfold file; re-run 6 November, three sevenfold files added to the `ES8SYM` pipeline 7 November)
Pairwise different order types	exact computation (convex-subset counts)
Real sets satisfy the formulas; cross-encoder agreement	exact computation, used as validation only
Random and brute-force tests; calibration runs; timings	evidence only (calibration proofs certified)
Rotations of order m other than by $2\pi/m$; affine symmetries (Corollary 1.2, Remark 1.3)	written proof (not refereed), new in this draft, not reviewed; the affine conjugation step is Lean-checked only in an order-forgetting form; for $m = 6$ the rotation by $2\pi \cdot 5/6$ is also covered by the Lean route via encoder B (`EncB.Geo.sixfold_es8_order6_of_texts`)
Absence of prior work	evidence only (bounded literature search)

Table 7: Rigor of the ingredients of Theorem 1.1.

Encoder B in Lean. The package `ENCBLEAN` has two parts. Its part `ENCBLEAN/EncB` (Lean 4.24.0 with its standard library only; milestones M0–M3 and M9) defines encoder B's formula as a Lean list of DIMACS clauses, `EncB.encB`, with parameters m, K, t, and the 64 sign patterns of the variables $1, 2, 3, 10, 11, 18$ as the list `EncB.cubes64`. Its part `ENCBLEAN/geo` (Lean 4.24.0 with Mathlib at commit `f897ebcf`, the commit of the repository; milestones M4–M6 and M8, committed in `862d95ed5`; the planned milestone M7, a reflection step, was dropped because the 64 cubes make it unnecessary) adds the geometric half and the assembly. It contains byte-identical copies of the repository modules it needs and of the Lean file `LEANEXT/capstone/Capstone2.lean`, which contains the text of `LAYERLEAN/LayerLemma.lean` apart from its import lines (Section 3), together with a Lean classification of phase patterns. The axiom check reports only `propext`, `Classical.choice` and `Quot.sound` for every theorem named here (`ENCBLEAN/logs/m9_d_recheck.log`, `ENCBLEAN/logs/m9_review_fix.log`, `ENCBLEAN/logs/m8_fix_audit.log`; for the names beginning with `EncB.Geo`, pinned by `#guard_msgs` in `ENCBLEAN/geo/ES8Geo/Audit.lean`). Lean-checked:

- the cube cover: if, for each of the 64 cubes, `EncB.encB` at $(6, 8, 11)$ together with the cube's six unit clauses is unsatisfiable, then so is `EncB.encB` at $(6, 8, 11)$ (`EncB.encB_unsat_of_cubes64`, from the general `EncB.not_dimacsSatisfiable_of_cover` and `EncB.covers_signPatterns`; `ENCBLEAN/EncB/Cover.lean`);
- soundness with respect to a combinatorial interface: for even $m \geq 2$ and every K, t, every `EncB.Model` with these parameters gives a satisfying assignment of `EncB.encB` (`EncB.encB_satisfiable`; `ENCBLEAN/EncB/Satisfiable.lean`). An `EncB.Model` consists of an orientation of each triple of labels and an order of the layer phases, together with, as hypotheses, the properties that the clause families use: the orientation is alternating and invariant under the rotation, the phase order is a strict total order with layer 0 first, the sign rules A and B hold on the circular order of the points and their antipodes, the nesting relations hold, and there is no chain of $K - 1$ points around a pivot;
- the reading of the printed text: the Lean string that the package's printer function computes for `EncB.encB` at $(6, 8, 11)$ (which the compiled program writes out, a step that is not Lean-checked), alone or with one of the 64 cubes, is parsed back to exactly that clause list by a DIMACS parser written in the package (`EncB.parseDimacs_render`, `EncB.parseDimacs_cube64_get`), so that unsatisfiability of the 64 parsed texts gives the hypothesis of the cube cover (`EncB.cubesUnsat64_of_texts_get`; `ENCBLEAN/EncB/Render.lean`);
- the geometric half (`ENCBLEAN/geo`): for even $m \geq 2$ and $K \leq 2m$, a set T of mt points in general position, invariant under the rotation by $2\pi/m$ about a point $c \notin T$, with no convex K-gon, has a polar labelling of its peeled layers (`EncB.Geo.labelling_of_counterexample`); for $m \geq 3$ and $K \geq 4$ the orientations of the labelled points and the order of the layer phases form an `EncB.Model` (`EncB.Geo.modelOfLabelling`), including the sign rules A and B on the real circular order of the directions;
- the assembly (`ENCBLEAN/geo/ES8Geo/Main.lean`): `EncB.Geo.sixfold_es8_of_texts` states that if each of the 64 texts of the third item parses, by the package's parser, to an unsatisfiable clause list, then every set S of at least 61 points in general position that is invariant under the rotation by $2\pi/6$ about some point contains 8 points in convex position (`HasConvexNGon 8 S`); `EncB.Geo.sixfold_es8_order6_of_texts` states the same for invariance under the rotation by $2\pi/6$ or by $2\pi \cdot 5/6$. Its proof composes the items above, through `EncB.no_model_6811_of_texts` (`ENCBLEAN/EncB/Compose.lean`) and `EncB.Geo.sixfold66_of_no_model`, with `sixfold_octagon` of Section 3. Its hypothesis is equivalent to the unsatisfiability of the 64 cube formulas of `EncB.encB` (`EncB.Geo.texts_iff_cubesUnsat64`), so a failing parse cannot make it hold trivially; it says exactly what the 64 verdicts are taken to certify.

The statements and the axioms of these theorems are fixed by `#guard_msgs` in `ENCBLEAN/geo/ES8Geo/Audit.lean`, so the build fails if one of them changes. The read-only script `ENCBLEAN/geo/scripts/audit_geo.sh` checks in addition the hashes of the copied files, of the definitions of `ENCBLEAN/EncB` and of the build configuration (with no Lean options that could weaken checking), the absence of `sorry`, added axioms, `native_decide` and similar escapes, and, by a text check only, that no tactic or evaluation is applied near a mention of the instance $(6, 8, 11)$ and that the declarations mentioning it are terms (the real protection against evaluating the formula is the time limit of the next step), and it elaborates the new files again, each within 300 s; it ends with `AUDIT PASS` (`ENCBLEAN/logs/m8_fix_audit.log`), and its self-test, which plants 22 violations in a copy, passes 23 of 23 cases (`ENCBLEAN/logs/m8_fix_selftest.log`).

So Theorem 1.1(a) for $m = 6$ is formally verified end to end via encoder B, assuming the 64 `cake_lpr` verdicts and the trust items (E1)–(E6) of gap (G2) in Section 5.10, summarised under "What is trusted" above. In particular these are not Lean-checked: that the printed texts are the certified files (their SHA-256, computed from the output of the compiled printer, equals the record for the base formula and all 64 cube files, 65 of 65, 68 of 68 with three further instances; `ENCBLEAN/scripts/hash_binding.sh`), which trusts the Lean compiler and runtime and a reading of the printer's input/output code (E4); the verdicts of `cake_lpr` (trusted: its verified HOL4/CakeML checker [32]); and that its parser reads these files as the package's parser does (not verified; tested on samples only, `ENCBLEAN/logs/m9_a2_cake_parse.log`). Checked by running compiled code, and not needed on this route: that `EncB.encB` equals the output of `gen_b2.py` (equal bytes, or a refusal where `gen_b2.py` raises, on all 225 instances of a small grid, and equal bytes on the recorded instances $(6, 8, 9)$, $(7, 8, 9)$, $(7, 8, 10)$ and $(6, 8, 11)$). Not covered: the sevenfold bound (`EncB.encB_satisfiable` needs even m; for $m = 7$ the written proof also needs the perturbation of Lemma 5.11(b)), encoder A, the lower bounds and the affine symmetries of Remark 1.3.

Further limitations. (1) The proof files (1.7 GB and 1.6 GB for $m = 7$; 41.0 GB and 46.5 GB for $m = 6$) are in local archives (`es8-c7-octagon-2026-11-03` and `es8-c6-octagon-2026-11-03`; the formulas regenerated for the replay of 4 November are in `es8-c6c7-review-2026-11-04`) and are not public, and so are the proofs of the sixfold replay of 6 November (28.4 GB compressed) and those of encoder B's cubes with $\neg Y(1, 2)$ (14.9 GB compressed, in `c6upper-noty12-20261106`); no public deposit with a DOI exists yet. The formulas can be regenerated, and solved and checked again, with the commands of Section 9 (a new solver run need not reproduce the same proof file; in the replay of 6 November, with CaDiCaL 2.1.3, 24 of the 64 proofs differed from the archived ones). (2) Some program comments in `C7UPPER/gen_b.py` and the angular docstring of `C7UPPER/gen.py` still state chain implications in the reverse direction, and that docstring omits the antipodal case of Section 5.8; the emitted formulas are unaffected (documentation only). (3) Nothing has been refereed.

9 Reproducibility

All commands run from `cloud/es7-2026-10-19/results`; tool locations are passed as environment variables.

Lean. From the repository root (dependencies prebuilt; 23 s and 2.6 GB on 6 November):

```
lake env lean cloud/es7-2026-10-19/results/LAYERLEAN/LayerLemma.lean
```

The Lean route for the sixfold bound via encoder B (Section 8), from `ENCBLEAN/geo` (its `lake-manifest.json` pins Mathlib `f897ebcf`; do not run `lake update`), and the hash binding, from `ENCBLEAN` after `lake build` there (about 15 minutes; it checks 68 hashes: the base formula, the 64 cube files and three further instances; on the machine of the runs it ran with `TMPDIR` set to a scratch directory under a 4 GB memory cap, `ENCBLEAN/README.md`):

```
lake exe cache get; lake build     # Mathlib oleans (network), then the package
scripts/audit_geo.sh               # read-only; must print AUDIT PASS (about 47 s)
cd ..; lake build                  # the Std-only package ENCBLEAN and its printer
scripts/hash_binding.sh            # exit 0 iff all 68 hashes (base + 64 cubes + 3 others) match
```

Sevenfold formulas and proofs.

```
python3 C7UPPER/gen.py 7 8 10 m7K8t10.cnf                # sha 1a7c0e70...; meta file names H(1) = var 7857
python3 C7UPPER/addunits.py m7K8t10.cnf m7K8t10_H1.cnf 7857   # appends H(1): sha fa297b5b...
python3 C6UPPER/encb/gen_b2.py 7 8 10 B_m7K8t10.cnf      # sha b4d29017... (= C7UPPER/gen_b.py output)
cadical --lrat --binary=false <cnf> <proof.lrat>         # CaDiCaL 3.0.1
cake_lpr --CML_HEAP_SIZE=8000 <cnf> <proof.lrat>         # must print: s VERIFIED UNSAT
S=<dir with resolve/bin/{cadical,lrat-check}> bash C7UPPER/repro.sh [solve]
```

`C7UPPER/repro.sh` runs the validation suite and, with `solve`, re-solves; its own proof check uses `lrat-check`, so `cake_lpr` must be run separately. Its record `C7UPPER/logs/repro_validation.log` (about 90 s according to `C7UPPER/README.md`; the log itself records no time) was produced in the original host layout. On 6 November 2026 its paths were adjusted for a checkout of the repository. A first validation run stopped at a 2-minute

limit (C7UPPER/logs/repro_validation_20261106.log). A second run of the adjusted script, without solve and with an absolute S, completed in 2.6 minutes on a fully loaded machine, with output equal to the original record apart from the paths of the point sets (C7UPPER/logs/repro_validation_20261106b.log). The solve branch was not re-run.

Sixfold formulas, cubes and proofs.

```
python3 C7UPPER/gen.py 6 8 11 m6K8t11.cnf                              # A: sha 542c8a3a...
python3 C6UPPER/tools/mkcubes.py m6K8t11.cnf m6K8t11.cnf.json cubes    # full_Y12.cnf + 32 cubes
python3 C6UPPER/tools/verify_cover.py m6K8t11.cnf m6K8t11.cnf.json cubes
python3 C6UPPER/encb/gen_b2.py 6 8 11 B_m6K8t11.cnf                    # B: sha 479e31a9...
python3 C6UPPER/tools/mkcubes.py B_m6K8t11.cnf C6UPPER/encb/B_m6K8t11.meta.json cubesB
P=<work dir> S=<dir with resolve/bin> python3 C6UPPER/tools/run.py 8 <deadline>   # CaDiCaL lanes
P=<work dir> S=<dir with resolve/bin> bash C6UPPER/tools/cake.sh                  # cake_lpr, one at a time
python3 -I C6C7REVIEW/replay.py --seed 1319527528443935447 ...          # the 4 November sampled replay
cp C6UPPER/replay-2026-11-06/replay*.py <dir>; cd <dir>          # the 6 November hash-bound replay
python3 replay.py          # first set R, CAKE, CAD at the top; or: LANES=6 python3 replay_par.py
# replay_par.py: env CAKE = parallel cake_lpr runs; delete its CAMP_CAP block (host-specific; README)
cp C6UPPER/noty12-2026-11-06/*.py <dir>; cd <dir>     # encoder B's 32 cubes with -Y(1,2) (6 November)
LANES=4 python3 noty12.py; python3 verify_final64.py  # set the paths at the top first; must print PASS
```

C6UPPER/tools/cake.sh runs cake_lpr only on proofs that lrat-check accepted; for an independent replay run cake_lpr on every proof, as the replay scripts of 6 November do (their README gives the details). noty12.py and verify_final64.py read the encoder-B base formula and cubes regenerated by the replay (its B/ directory), and they name the tool binaries and these paths at the top of each script.

Lower bounds and exact checks.

```
python3 SIX60/verify/selftest.py; python3 SIX60/verify/verify60.py SIX60/sets/c6_60pts_D.pts
gcc -O2 -o dfs_convex SIX60/verify/dfs_convex.c          # input: oblique integer pairs, see SIX60/README.md
sh PRIMEFOLD/check64/run_all.sh                          # ~35 s; rewrites check64/check64.log in place
sh ES8SYM/tools/run_all.sh ES8SYM <outdir>               # the ES8SYM pipeline (about one minute)
bash K8EXACT20261106/run.sh <scratch dir>                # exact clause check, ~2-3 min at one core
python3 -I WRITEUP/P1/tools/make_tables.py               # regenerates and checks the tables of this paper
```

Run times and machines. Recorded times are in Sections 6 and 7. The sevenfold cake_lpr logs are in the format of the macOS time utility; the hardware of each original run is not uniformly recorded in the logs. The re-checks of 6 November ran on a Linux workstation with 32 hardware threads and about 64 GB of memory (59.5 GiB visible to Linux), under per-process memory caps; the sixfold replay of 6 November, and the run for encoder B's cubes with $\neg Y(1,2)$, ran on the same workstation.

10 Open problems

The other rotation orders at $k = 8$ (Table 1). By the layer lemma, a counterexample to $ES(8) = 65$ with fivefold symmetry needs 13 layers (65 points, with or without centre), and one with fourfold symmetry 17 layers (68 points). The encodings of Section 4 apply unchanged, but the threshold formulas have not been refuted. The following is evidence only; timed-out runs prove nothing. For $m = 5$, splitting by the 64 sign patterns of $Y(1,2), \ldots, Y(3,4)$ with the unit $H(1)$ refutes the 40 patterns that contradict transitivity at once, but phase-order cubes ran for up to 22,102 s (about $5 \cdot 10^7$ conflicts, against 2.0–$2.6 \cdot 10^6$ for the sixfold cubes) without an answer (ROT456 §4). Runs of 5 November with additional clauses from a chord lemma (written proof, not refereed; each added clause checked by exact computation; CHORD20261105) refuted the impossible patterns again, with proofs accepted by cake_lpr, but closed none of the 24 fivefold phase orders (19 ran for up to about 15 hours, 5 were never started) and none of the 12 fourfold ones (about 15 hours; one run was lost) (ROT45CHORD20261105). At 14 fourfold layers an abstract model consistent with all 3-term Grassmann–Plücker (GP) relations exists and has not been realized (ROT456 §3). The plain fivefold encoding is satisfiable at 12 layers; after lazy GP refinement a GP-consistent 12-layer model was found and then excluded for the prescribed fivefold rotation by an exact linear identity over $\mathbb{Q}(\sqrt{5})$ (written proof, not refereed, with the identity checked by exact computation; ROTREFINE20261104). With such identities

added the 12-layer formula stayed satisfiable, by models that are not GP-consistent, and the refined 13-layer formula gave no answer in 600 s (`ROTREFINE20261104`). *Is every fivefold symmetric set of* 65 *points, and every fourfold one of* 68 *points, in general position forced to contain* 8 *points in convex position?*

Threefold symmetry. Here $8 > 2 \cdot 3$ and the layer lemma does not apply. Every convex layer is a triangle, a hexagon or the centre (written proof, not refereed, `ES8SYM` §3), and since $3 \cdot 21 + 1 < 65$, a threefold counterexample to $ES(8) = 65$ exists if and only if some threefold set of 22 orbits without centre is octagon-free. The largest known threefold octagon-free sets have 61 points; a threefold 64-point set and its 63-point subset without the centre each have exactly three convex octagons (exact computation, `ES8SYM`). A radius-ordered relaxation of the encoding without nesting clauses (written soundness argument, not refereed, `C6C7REVIEW`, `THREEFOLDRELAX`) was implemented as `THREEFOLDRELAX/gen_relax.py` with the option `-nonest` and validated on the known threefold sets (exact computation). It is too weak (evidence only): for $K = 7$ it stays satisfiable at 10, 11, 12 and 13 orbits, although threefold sets without 7 points in convex position have at most 10 orbits (computer-assisted: the threefold 33-point statement `Threefold33` of `SYMLEAN`, certified in `C3FINAL`, by `cake_lpr` verdicts recorded at solve time on hash-matched formulas; the proofs were not kept, and 2,120 of the 116,883 pieces were re-solved and accepted again, `C3SPOT`), and the threshold formula $(3, 8, 22)$, with 170,270 variables and 7,400,332 clauses, gave no answer in 800 s.

The half-turn. Does every centrally symmetric set of 66 points in general position contain 8 points in convex position? 64-point examples without one exist (Section 1.1).

Larger k. SMQH's suspicion [28, §7] asks for symmetric extremal sets for all $k \geq 7$. For $(k-1)$-fold symmetry the size 2^{k-2} is possible only if $k - 1$ is a power of 2, an odd prime or an odd base-2 Fermat pseudoprime (orbit counting; core Lean-checked in `PRIMEFOLD/compat`; for $k = 7$, SMQH [28, §6.2] already note that a 7-gon-free s-fold symmetric 32-point set needs $s \in \{1, 2, 4\}$); for $k = 8$ the sevenfold sets above attain it. For $k = 9$ the largest eightfold set without 9 points in convex position that we know has 120 of the 128 points that $ES(9) = 129$ would allow (`PRIMEFOLD` §5; exact computation).

Formalization. For the sixfold bound via encoder B, gap (G1) is closed in Lean and gap (G2) is reduced to its trust items (E1)–(E6) (Section 8, "Encoder B in Lean"): `EncB.Geo.sixfold_es8_of_texts` goes from the unsatisfiability of the 64 printed cube texts to the geometric statement, in the spirit of the Lean-verified encoding of the empty hexagon theorem [27], and what stays trusted is the 64 `cake_lpr` verdicts and the items (E1)–(E6). Open are: the sevenfold bound in Lean (encoder B's soundness for odd m, which in the written proof needs the perturbation of Lemma 5.11(b); `EncB.encB_satisfiable` needs even m); a verified DIMACS parser, or a proof that `cake_lpr`'s parser agrees with the Lean parser on these files, to remove (E3); and public proof files, so that others can re-check the 64 verdicts (E2) (two have been reproduced with `ENCBLEAN/scripts/resolve.sh`; re-solving all 64 is estimated at about 22 hours and was not run). Encoder A has no Lean counterpart.

Acknowledgements

[acknowledgements, and a statement on the use of computational and AI assistance: to be written by the author]

A Hashes

SHA-256 of the files on which Theorem 6.1 rests. The first table gives the programs, the base formulas and the two sevenfold proofs (from `C6UPPER/SHA256SUMS.txt`, `C7UPPER/SHA256SUMS.txt` and the Lean file; the program files were re-hashed for this draft). The sixfold cube certificates are the proofs of the hash-bound replay of 6 November (Section 6.2). The hashes of their formulas, proofs and compressed proofs are in `C6UPPER/replay-2026-11-06/results.tsv`, and the tool hashes in `TOOLS_AND_PROOFS.sha256` there. The 64 cube formulas have the hashes recorded in `C6UPPER/SHA256SUMS.txt`. For the 40 contradictory

cubes the replay proofs are byte-identical to the archived proofs of 3 November, whose hashes are also in `C6UPPER/SHA256SUMS.txt`. For the 24 phase-order cubes the replay proofs are new files; the second table gives the first twelve digits of their hashes (Table 4 gives those of the proofs of 3 November). For encoder B's 32 cubes with $\neg Y(1,2)$ the certificates are the proofs of the run of 6 November; the hashes of their formulas, proofs and compressed proofs are in `C6UPPER/noty12-2026-11-06/results.tsv`, the formula hashes also in `cubes.sha256` there (written before solving), and the tool hashes in `TOOLS_AND_PROOFS.sha256` there.

file	SHA-256
LAYERLEAN/LayerLemma.lean	`afb8542dceb0c17005b4eccec1b79abbb57099c0768e1a4bbb1cf52ad797257f`
C7UPPER/gen.py (encoder A)	`4564e7f0ff1d3794917414002b4847cca6f0753420bd806af9d107f2a361bace`
C7UPPER/gen_b.py (encoder B, odd m)	`8f09632fa22888b6feca34e729e3b74d25905eef2a6c4058934298075af96398`
C6UPPER/encb/gen_b2.py (encoder B)	`3d47459a9723b5f7344205f3f2d7463e0be00b41c5070155e5da8dbf78fd40ba`
C7UPPER/addunits.py	`c6654fa5f1a26ec76b0aeaddb584ae84f87c565e33dc60f6a8f1c13f4787eecc`
C6UPPER/tools/mkcubes.py	`9b99b7003103d47b41cd2267b40efd34c77c44e35050f891e2f02c27953af395`
C6UPPER/tools/verify_cover.py	`f2780145e680e2fccc703ecdd6d9b2667eba8aa7965413d6090ec6dbeaabd914`
A, $(6,8,11)$: `m6K8t11.cnf`	`542c8a3a2c4484a7d60b5fdb8efc0f3506d782031fc4b4625604b360211ad51e`
A, $(6,8,11)+Y(1,2)$: `full_Y12.cnf`	`1ea25d88ffe81086b6c44871dc1ad6d7489d9abc222717ec2394c819c71e09af`
B, $(6,8,11)$: `B_m6K8t11.cnf`	`479e31a9e8d58d752c86b9abf02fb799c10615a7e61a88f132c49734674f37fc`
B, $(6,8,11)+Y(1,2)$: `B/full_Y12.cnf`	`1b09c21076189717407dfb4e187bbaa41fd2174f8b7eb7a5d61f7e5af2153395`
A, $(7,8,10)$: `cnf/m7K8t10.cnf`	`1a7c0e706f2a7fb52b109ae20ca37d49c74940e01c1227fe152b46d742d5997f`
A, $(7,8,10)+H(1)$: `cnf/m7K8t10_H1.cnf`	`fa297b5bd6608d29369fd56f8cab20d6d17ad9d335253352541a42e4c669fc49`
B, $(7,8,10)$: `cnfb/m7K8t10.cnf`	`b4d290176cb675c61ecbcf70380cc520aabf093d00e2572328a0975ae2d296e2`
LRAT, A $(7,8,10)+H(1)$	`c750a805a7a3dbdb4ba445c16fb75051a9fac623a3ddf93a5d864defe41039b2`
LRAT, B $(7,8,10)$	`75f9bd350186724a85e949b05315369d88ef887c9953e0048810b67e8172550e`

cube	encoder A	encoder B	cube	encoder A	encoder B
`c00000`	`4774604b24ff`	`ecea6c27ccce`	`c11000`	`9367a488c2d8`	`cacc217c2175`
`c00001`	`ae948f6e8d3d`	`c6ce85da6a84`	`c11001`	`2138536bd98b`	`4fc2c10931cb`
`c01001`	`aa64afa704cc`	`41775291e205`	`c11011`	`2fca62e299bf`	`f0598e6b8302`
`c01011`	`d3819218999a`	`0381e5d5e7cf`	`c11100`	`eddc6c053b50`	`7da1e009e075`
`c10000`	`4ef2cfb0d91e`	`5b7f8c34d5dc`	`c11110`	`c72fd8d440f0`	`a24719abd74b`
`c10100`	`9d8d418b6cdc`	`afbc5c89a507`	`c11111`	`b666f7c34eef`	`b3cc4b515d16`

B Seeds of the point sets

Each set is the union of the orbits $\{\zeta^j s : 0 \le j < m\}$ of the listed seeds s, written as coefficient vectors $(w_0, w_1, \dots)$ of $\sum_j w_j \zeta^j$ with trailing zeros omitted, plus the centre for $m = 7$. Coordinates are scaled integers; only orientations matter. The listing is generated from the files by `WRITEUP/P1/tools/make_tables.py`, which checks the hashes and that the rows other than the centre are exactly the m cyclic shifts of the listed seeds (the seed of an orbit is its first row in the file).

Sixfold, $\zeta = e^{2\pi i/6}$.

A `SIX60/sets/c6_60pts_A.pts` (SHA-256 `614469e928dbc68c...`; scale 2^{28}): $(268435456, 0)$, $(74257719, 6629426)$, $(65544021, 6163452)$, $(54071730, 10588904)$, $(53501118, 10593050)$, $(43264300, 15650877)$, $(42748054, 15582152)$, $(29453840, 15103124)$, $(29453745, 15103180)$, $(2569808, 3435)$.

B `SIX60/sets/c6_60pts_B.pts` (SHA-256 `2a0c981e6e240207...`; scale 2^{28}): $(268435456, 0)$, $(84911, 75878)$, $(82623, 75568)$, $(69289, 76753)$, $(68638, 76095)$, $(60816, 78578)$, $(60324, 78009)$, $(53911, 72346)$, $(50938, 73485)$, $(225, 2559)$.

C `SIX60/sets/c6_60pts_C.pts` (SHA-256 `61ed920d90debaee...`; scale 10^{12}): $(1000000000000, 0)$, $(4171582596, 4220918302)$, $(3901289731, 4222104075)$, $(3775207400, 4222708268)$, $(2106585128, 2441364552)$, $(1983448959, 2441678397)$, $(1939873218, 2455394490)$, $(1895765476, 2420505141)$, $(1812312622, 2442140453)$, $(2641721, 3611115)$.

C′ `SIX60/sets/c6_60pts_C2.pts` (SHA-256 `8ba0b2e964dc5135...`; scale 10^9): $(1000000000, 0)$, $(4171583, 4220918)$, $(3901290, 4222104)$, $(3775208, 4222708)$, $(2106585, 2441365)$, $(1983449, 2441678)$, $(1939873, 2455395)$, $(1895766, 2420505)$, $(1812313, 2442140)$, $(2642, 3611)$.

D `SIX60/sets/c6_60pts_D.pts` (SHA-256 `e2eaceb9e6483b46...`; scale 2^{19}): $(524288, 0)$, $(131119, 11665)$, $(131103, 11664)$, $(107744, 21423)$, $(107703, 21423)$, $(88128, 30720)$, $(86762, 30677)$, $(56942, 30072)$, $(56913, 30089)$, $(5569, 389)$.

Sevenfold, $\zeta = e^{2\pi i/7}$.

`c7_64pts_small`
(SHA-256 `ebe028f65c3856a9...`; coefficients of at most 3 digits): the centre and $(661, -486)$, $(30, 179)$, $(211, -32)$, $(220, -206)$, $(55, -179)$, $(164, -37)$, $(164, -163)$, $(42, 102)$, $(-110, -29)$.

`c7_64pts_9layers_centre`
(SHA-256 `23e1389d3d125861...`; coefficients of at most 12 digits): the centre and $(509924072692, -375298680295)$, $(22983119307, 138080130704)$, $(162614727650, -24500525116)$, $(169787658298, -158878267967)$,

(42812980169, −138092695649), (126370669637, −28418729360), (126408891754, −125673507785), (32253718886, 79162223272), (−84612764862, −22144839158).

`c7_64_own_s1000`
(SHA-256 `c9942cc187fea8f5...`; coefficients of at most 4 digits): the centre and (−6004, −7770), (312, −928), (−531, −278), (−585, 891), (−417, 653), (−396, −150), (148, −581), (−377, 613), (125, −545).

`c7_64pts_9layers_centre_own`
(SHA-256 `ec48146993b86fc2...`; coefficients of at most 6 digits): the centre and (−600412, −777042), (31148, −92791), (−53081, −27821), (−58504, 89131), (−41694, 65250), (−39573, −15020), (14799, −58050), (−37711, 61349), (12463, −54456).

`p7_compound_64`
(SHA-256 `ef50ee00829d9781...`; coefficients of at most 14 digits; rows not in layer order): the centre and (9999997444195, 1528880), (9999998722117, 763928), (10000000000078, −2048), (10000001277883, −763928), (10000002555844, −1529904), (10000003833766, −2294856), (9997439504092, 2014452945), (9997440780000, 2013730000), (9997442055816, 2013008110).

`p7_two_orbit_64`
(SHA-256 `9a55f2451b1427ba...`; coefficients of at most 13 digits; rows not in layer order): the centre and (999975261620, 10374155), (999987630972, 5186493), (1000000000648, −2338), (1000012369028, −5186493), (1000024738704, −10375324), (1000037108056, −15562986), (940387290144, 47106798916), (940400000000, 47100000000), (940412709712, 47093202168).

`rule_p7_64pts`
(SHA-256 `bb38d29ae9561720...`; coefficients of at most 13 digits): the centre and (1019085068556, −17047921694), (1018993117909, −16975856542), (1018988590639, −16974272382), (1017225474082, −15494375528), (1017137250875, −15463671857), (1017132606475, −15461712783), (982796372673, 15928181419), (981031908534, 16542254841), (980939020533, 16581436326).

`m7_x2_G9s_centre`
(SHA-256 `ac524df7a5a62bed...`; coefficients of at most 31 digits; rows not in layer order): the centre and (99999999999999998500000000000, −39500000000, 15000000000000), (99999999999999999100000000000, −35300000000, 9000000000000), (99999999999999999200000000000, −74800000000, 8000000000000), (10000000000000000030000000000000, −52100000000, −3000000000000), (10000000000000000080000000000000, −55200000000, −8000000000000), (10000000000000000120000000000000, 74800000000, −12000000000000), (10000000000000000130000000000000, 46100000000, −13000000000000), (10000000000000000140000000000000, 35400000000, −14000000000000), (10000000000000000150000000000000, 31900000000, −15000000000000).

References

[1] J. Baek, On the Erdős–Tuza–Valtr conjecture, *European J. Combin.* 124 (2025), 104085; arXiv:2206.04260. Theorem numbers refer to the arXiv version.

[2] J. Baek and M. Balko, The Erdős–Szekeres conjecture revisited, in *SoCG 2025*, LIPIcs 332 (2025), 13:1–13:15, doi:10.4230/LIPIcs.SoCG.2025.13. Journal version: *J. Combin. Theory Ser. A* 222 (2026), 106195 (not read).

[3] M. Balko and P. Valtr, A SAT attack on the Erdős–Szekeres conjecture, *European J. Combin.* 66 (2017), 13–23.

[4] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks and F. Pollitt, CaDiCaL 2.0, in *Computer Aided Verification (CAV 2024)*, LNCS 14681, Springer, 2024, 133–152.

[5] L. Cruz-Filipe, M. J. H. Heule, W. A. Hunt Jr., M. Kaufmann and P. Schneider-Kamp, Efficient certified RAT verification, in *CADE-26*, LNCS 10395, Springer, 2017, 220–236.

[6] L. de Moura and S. Ullrich, The Lean 4 theorem prover and programming language, in *CADE-28*, LNCS 12699, Springer, 2021, 625–635.

[7] drat-trim, `https://github.com/marijnheule/drat-trim`: issue #31 (22 July 2022), commits `cbd2915` (19 October 2022, fix) and `5cd8ac3` (21 April 2024, regression), issue #42 and pull request #43 (29 September 2026); master at `2e3b2dc`.

[8] B. Dumitru, Notes on the 33-point Erdős–Szekeres problem, arXiv:2512.24061 (2025).

[9] P. Erdős and G. Szekeres, A combinatorial problem in geometry, *Compositio Math.* 2 (1935), 463–470.

[10] P. Erdős and G. Szekeres, On some extremum problems in elementary geometry, *Ann. Univ. Sci. Budapest. Eötvös Sect. Math.* 3–4 (1960/61), 53–62.

[11] P. Erdős, Zs. Tuza and P. Valtr, Ramsey-remainder, *European J. Combin.* 17 (1996), 519–532, doi:10.1006/eujc.1996.0045 (not read).

[12] B. Georgiev, J. Gómez-Serrano, T. Tao and A. Z. Wagner, Mathematical exploration and discovery at scale, arXiv:2511.02864v3 (2025).

[13] M. J. H. Heule and M. Scheucher, Happy ending: an empty hexagon in every set of 30 points, in *TACAS 2024*, LNCS 14570, Springer, 2024, 61–80, doi:10.1007/978-3-031-57246-3_5.

[14] A. F. Holmsen, H. Nassajian Mojarrad, J. Pach and G. Tardos, Two extensions of the Erdős–Szekeres problem, *J. Eur. Math. Soc.* 22 (2020), 3981–3995.

[15] J. D. Kalbfleisch, J. G. Kalbfleisch and R. G. Stanton, A combinatorial problem on convex n-gons, in *Proc. Louisiana Conf. Combinatorics, Graph Theory and Computing*, Baton Rouge, 1970, 180–188.

[16] V. Koshelev and A. Koshka, Combinatorial geometry of Erdős–Szekeres type problems: SAT/ASP modeling and linear subreduction, arXiv:2604.20120 (2026).

[17] A. Krapivin, B. Przybocki and M. J. H. Heule, Toward satisfiability modulo realizability, arXiv:2607.02958 (2026); to appear in *CICM 2026* (arXiv comment; no proceedings record found on 6 November 2026).

[18] E. Mackey, *Pinwheels and Polygons: Symmetric Realizations of Polygon-Free Point Placements via SAT*, M.S. thesis, Carnegie Mellon University, Technical Report CMU-CS-25-108 (2025).

[19] F. Marić, Fast formal proof of the Erdős–Szekeres conjecture for convex polygons with at most 6 points, *J. Automat. Reason.* 62 (2019), 301–329.

[20] The mathlib Community, The Lean mathematical library, in *CPP 2020*, ACM, 2020, 367–381.

[21] W. Morris and V. Soltan, The Erdős–Szekeres problem on points in convex position – a survey, *Bull. Amer. Math. Soc.* 37 (2000), 437–458.

[22] H. Nassajian Mojarrad and G. Vlachos, An improved upper bound for the Erdős–Szekeres conjecture, *Discrete Comput. Geom.* 56 (2016), 165–180.

[23] M. Scheucher, Two disjoint 5-holes in point sets, *Comput. Geom.* 91 (2020), 101670, doi:10.1016/j.comgeo.2020.101670 (not read; cited as in [13, 28]).

[24] M. Scheucher, A SAT attack on Erdős–Szekeres numbers in $\mathbb{R}^d$ and the empty hexagon theorem, *Comput. Geom. Topol.* 2(1) (2023), 2:1–2:13, doi:10.57717/cgt.v2i1.12 (not read; cited as in [13]).

[25] M. Schroeder, Erdős–Szekeres: upper bounds for convex heptagons and octagons, archive version 1, 19 September 2026, doi:10.5281/zenodo.22842891 (unreviewed).

[26] B. Subercaseaux, J. Mackey, M. J. H. Heule and R. Martins, Automated mathematical discovery and verification: minimizing pentagons in the plane, in *Intelligent Computer Mathematics (CICM 2024)*, LNCS 14960 (LNAI), Springer, 2024, 21–41, doi:10.1007/978-3-031-66997-2_2 (not read; cited as in [28, §1]).

[27] B. Subercaseaux, W. Nawrocki, J. Gallicchio, C. Codel, M. Carneiro and M. J. H. Heule, Formal verification of the empty hexagon number, in *ITP 2024*, LIPIcs 309 (2024), 35:1–35:19.

[28] B. Subercaseaux, E. Mackey, L. Qian and M. J. H. Heule, Automated symmetric constructions in discrete geometry, arXiv:2506.00224v1 (2025); also in *Intelligent Computer Mathematics (CICM 2025)*, LNCS 16136 (LNAI), Springer, 2026 (published October 2025), 29–47, doi:10.1007/978-3-032-07021-0_3 (Crossref metadata; chapter text not accessed).

[29] A. Suk, On the Erdős–Szekeres convex polygon problem, *J. Amer. Math. Soc.* 30 (2017), 1047–1053.

[30] Supplementary material: the directories `LAYERLEAN`, `C6UPPER` (including `C6UPPER/replay-2026-11-06` and `C6UPPER/noty12-2026-11-06`), `C7UPPER`, `C6C7REVIEW`, `SIX60`, `PRIMEFOLD`, `ES8SYM`, `K8EXACT20261106`, `LRATCHECK20261105`, `CHORD20261105`, `ROT456`, `ROTREFINE20261104`, `ROT45CHORD20261105`, `SYMLEAN`, `SYM5`, `STATUS20261104`, `THREEFOLDRELAX`, `C3FINAL`, `C3SPOT`, `WRITEUP/note`, `WRITEUP/P1`, `NOVELTY20261106` and `LIT` under `cloud/es7-2026-10-19/results`, and, for the $ES(8) \le 311$ remark, `experiments/omission-final311-2026-10-14` and `experiments/omission-compact-checker-2026-11-05`, of the project repository, branch `es7-night-2026-11-01` (the records cited are those at commit `f2a462b67` or later; the commit to cite is to be fixed before circulation); proof archives `es8-c7-octagon-2026-11-03` and `es8-c6-octagon-2026-11-03`, the review archive `es8-c6c7-review-2026-11-04` (regenerated formulas, models, witnesses), the proofs of the sixfold replay of 6 November, `c6upper-replay-20261106`, and those of encoder B's cubes with $\neg Y(1,2)$, `c6upper-noty12-20261106` (not public). [A public deposit with a DOI, of the generators, cube files, logs, proof hashes, replay logs and, where size permits, the LRAT proofs, is to be made before circulation.]

[31] G. Szekeres and L. Peters, Computer solution to the 17-point Erdős–Szekeres problem, *ANZIAM J.* 48 (2006), 151–164.

[32] Y. K. Tan, M. J. H. Heule and M. O. Myreen, Verified propagation redundancy and compositional UNSAT checking in CakeML, *Int. J. Softw. Tools Technol. Transf.* 25 (2023), 167–184; conference version *cake_lpr: verified propagation redundancy checking in CakeML*, TACAS 2021, LNCS 12652, 223–241.